

Sign Language - Based Campus Navigation System Using Deep Learning

By

Liya Khatija
(2448033)

Under the guidance of
Dr. Dalvin Vinoth Kumar A

A Project report submitted in partial fulfillment of the requirements for the award of degree of Master of Science (Data Science) of CHRIST (Deemed to be University)
March - 2026

CERTIFICATE

This is to certify that the report titled ***Sign Language - Based Campus Navigation System Using Deep Learning*** *is a bona fide record of work done by* ***Liya Khatija (2448033)*** *of CHRIST (Deemed to be University), Bengaluru, in partial fulfillment of the requirements of 6th Trimester M.Sc. (Data Science) during the year 2025-26.*

Head of the Department **Project Guide**

Valued-by:

1.

2.

Name	: Liya Khatija
Register Number	: 2448033
Examination Centre	: CHRIST(Deemed to be University)
Date of Exam	:

ACKNOWLEDGEMENT

I would like to express my sincere and heartfelt gratitude to Dr. Nimitha John, Head of the Department, and Dr. Monisha Singh, Project Coordinator, for their constant support, encouragement, and guidance throughout the duration of this project. Their leadership and academic direction have been instrumental in providing a strong foundation for the successful completion of this work.

I am deeply indebted to my project guide, Dr. Dalvin Vinoth, for his invaluable guidance, continuous support, and constructive feedback at every stage of this project. His insightful suggestions, patience, and encouragement not only helped in shaping the direction of this work but also enhanced my understanding of the subject. His mentorship played a pivotal role in overcoming challenges and ensuring the successful completion of this project.

I would also like to extend my sincere thanks to the Department of Statistics and Data Science, CHRIST (Deemed to be University), for providing the necessary resources, infrastructure, and a stimulating academic environment. The exposure to advanced tools, technologies, and academic support greatly facilitated the smooth execution of this project.

I express my heartfelt appreciation to all the faculty members for their continuous guidance, support, and for imparting the knowledge and technical skills required to undertake this project. Their teachings and encouragement have significantly contributed to my academic growth and the successful execution of this work.

Finally, I would like to convey my deepest gratitude to my family and friends for their unwavering support, patience, and constant encouragement throughout this journey. Their motivation and belief in me have been a source of strength and inspiration during this project.

ABSTRACT

This project presents a deep learning-based sign language recognition system designed for structured campus navigation. The system focuses on recognizing a predefined set of nine gesture classes corresponding to campus locations and mapping them to navigation instructions. The problem is formulated as a closed-set multi-class classification task using video input sequences. The initial phase employs a landmark-based approach using MediaPipe to extract 3D hand keypoints, followed by temporal modeling using Bidirectional LSTM and CNN–BiLSTM architectures. Feature engineering techniques, including joint angle and velocity-based motion features, are incorporated to enhance representation, while targeted data augmentation is applied to address class imbalance. In the second phase, a YOLOv8-based hand detection module is introduced to improve spatial localization. A hybrid pipeline combining detection, ROI extraction, landmark processing, and temporal classification is developed. Experimental results demonstrate that data balancing and geometric feature engineering significantly improve performance in small datasets. While the landmark-based model achieves moderate accuracy, the detection-enhanced pipeline shows substantial improvement in robustness and classification performance. The study highlights the importance of dataset quality, feature representation, and modular system design in developing practical assistive solutions for sign-based navigation.

TABLE OF CONTENTS

Acknowledgements ..... **iii**

Abstract ..... **iv**

List of Tables ..... **ix**

List Of Figures ..... **x**

Abbreviations ..... **xi**

1. Introduction 1

    1.1 Background and Motivation 1

    1.2 Existing System 1

    1.3 Problem Statement 3

    1.4 Objectives 3

        1.4.1 Landmark-Based Modeling Objectives 3

        1.4.2 Yolo-Based Detection Objectives 3

    1.5 Scope of the Project 5

2. Literature Review 6

3. System Design 9

    3.1 System Overview 9

        3.1.1 Landmark-Based Pipeline 9

        3.1.2 Detection-Enhanced Pipeline 9

    3.2 System Architecture 10

        3.2.1 Architectural Characteristics 10

    3.3 Module Description 12

        3.3.1 Input Module 12

        3.3.2 Preprocessing Module 12

        3.3.3 Detection Module (YOLO) 12

        3.3.4 ROI Extraction Module 13

        3.3.5 Feature Extraction Module (MediaPipe) 13

        3.3.6 Feature Engineering Module 14

        3.3.7 Temporal Modeling Module 14

        3.3.8 Classification Module 14

3.3.9 Output Module....15
3.4 System Workflow 15
3.5 Design Considerations And Trade-Offs 17
4. Dataset And Preprocessing 19
4.1 Dataset Collection and Description 19
4.2 Frame Extraction and Temporal Standardization 20
4.3 Landmark Extraction and Feature Representation 21
4.3.1 Joint Angle Features 21
4.3.2 Velocity Features 22
4.4 Dataset Splitting and Imbalance Handling 22
4.5 YOLO-Based Detection Dataset Preparation 22
4.6 ROI Extraction and Hybrid Pipeline 23
4.7 CNN-Based Feature Extraction 25
5. Landmark-Based Temporal Modeling 26
5.1 Problem Formulation 26
5.2 Baseline BiLSTM Architecture 27
5.3 Training Strategy Optimization 27
5.4 Feature Engineering 28
5.4.1 Angle-Based Features 28
5.4.2 Velocity-Based Features 28
5.5 Handling Class Imbalance 29
5.6 CNN-BilSTM Hybrid Architecture 29
5.7 Experimental Exploration 30
5.8 Observations and Insights 30
5.9 Final Model Selection 31
6. YOLO-Based Hand Detection and Hybrid Pipeline 33
6.1 Rationale for Detection-Based Enhancement 33
6.2 Detection Dataset Construction 34
6.3 Dataset Annotation Using Roboflow 34
6.4 YOLOv8 Model Training 35
6.5 ROI Extraction Strategy 35

6.6 Dataset Splitting Strategy 36
6.7 CNN-Based Feature Extraction 37
6.8 Temporal Sequence Construction 37
6.9 Class Imbalance Handling 38
6.10 Integration Into Hybrid Pipeline 38
6.11 Design Trade-Offs 39
6.12 Key Insights 40
7. Results And Comparative Analysis 41
7.1 Landmark-Based Model Results 41
7.2 YOLO Hand Detection Results 51
7.3 ROI Extraction Impact 52
7.4 YOLO + CNN + BiLSTM Results 53
7.5 Final Comparative Summary 54
7.6 Key Scientific Findings 54
8. Implementation 55
8.1 Development Environment 55
8.2 System Execution Pipeline 56
8.3 ROI Extraction Implementation 56
8.4 Feature Extraction Using ResNET50 57
8.5 Temporal Buffer Implementation 57
8.6 BiLSTM Model Inference 58
8.7 Real-Time Prediction Mechanism 59
8.8 User Controls 59
8.9 Output Visualization 59
8.10 Sample Outputs 60
8.11 Implementation Challenges 61
8.12 System Validation 61
9. Discussion 63
10. Future Work 69
11. Conclusion 72
Appendices 74

Appendix A – User Manual 74
Appendix B – System Configuration.....75
Appendix C – Design Considerations and Analysis.....76
Appendix D – Sample Code Snippets…….....78
References 80

LIST OF TABLES

Table No.	Table Name	Page No.
Table 3.1	Data Representation Across Stages	10
Table 7.1	Comparison of Class-wise Recall Before and After Augmentation	45
Table 7.2	CNN + BiLSTM Performance	46
Table 7.3	CNN + BiLSTM with Angle Features	47
Table 7.4	CNN + BiLSTM + Velocity Features	49
Table 7.5	Landmark Model Comparison	54
Table 7.6	Detection vs Classification Comparison	54

LIST OF FIGURES

Fig No.	Figure Name	Page No.
Fig 3.1	Overall System Interaction Diagram	9
Fig 3.2	System Architecture of Proposed Model	11
Fig 3.3	Modular System Design	15
Fig 3.4	System Workflow	16
Fig 4.1	Dataset Collection and Preprocessing Pipeline	20
Fig 4.2	YOLO-Based Data Preparation Pipeline	24
Fig 5.1	Evolution of Landmark-Based Models and Feature Enhancements	32
Fig 6.1	Transformation Pipeline from Raw Frames to Gesture Prediction	39
Fig 7.1	Training and Validation Accuracy Curve (Baseline BiLSTM)	42
Fig 7.2	Training and Validation Loss Curve (Baseline BiLSTM)	42
Fig 7.3	Confusion Matrix of Baseline BiLSTM	43
Fig 7.4	Training and Validation Curves with Early Stopping	44
Fig 7.5	Confusion Matrix after Targeted Augmentation	45
Fig 7.6	Comparison of Class-wise Recall Before and After Augmentation	46
Fig 7.7	Precision–Recall Curve for YOLOv8	51
Fig 7.8	mAP Curve Across Training Epochs	51
Fig 7.9	Sample Hand Detection Outputs	52
Fig 7.10	Example of ROI Extraction After Cropping	52
Fig 7.11	Confusion Matrix of YOLO-Enhanced Pipeline	53

Fig 8.1	Real-Time Prediction Output for RDBlock Gesture	60
Fig 8.2	Real-Time Prediction Output for Office Gesture	60
Fig 8.3	Real-Time Prediction Output for Block4 Gesture	61

ABBREVIATIONS

Abbreviation	Full Form
AI	Artificial Intelligence
ML	Machine Learning
DL	Deep Learning
CNN	Convolutional Neural Network
LSTM	Long Short-Term Memory
BiLSTM	Bidirectional Long Short-Term Memory
YOLO	You Only Look Once
ROI	Region of Interest
mAP	Mean Average Precision
FPS	Frames Per Second
RGB	Red Green Blue
ReLU	Rectified Linear Unit

1. INTRODUCTION

1.1 BACKGROUND AND MOTIVATION

Sign language is a primary mode of communication for individuals with hearing and speech impairments. Despite advancements in assistive technologies, accessibility challenges continue to exist in structured environments such as university campuses. Navigation within a campus often relies on spoken communication, textual signage, or mobile interfaces that may not naturally align with sign-based interaction.

With the rapid growth of computer vision and deep learning, automated hand gesture recognition systems have become increasingly viable. Technologies such as MediaPipe enable real-time extraction of three-dimensional hand landmarks, while deep learning architectures such as Convolutional Neural Networks (CNNs) and Long Short-Term Memory (LSTM) networks allow modeling of spatial and temporal patterns in dynamic gestures. Additionally, object detection frameworks like YOLO provide strong capabilities for accurate hand localization in complex visual scenes.

However, many existing sign language systems focus on full sentence translation or large vocabulary recognition, which require extensive datasets and computational resources. In contrast, structured applications such as campus navigation require reliable recognition of a limited set of predefined gestures corresponding to specific destinations.

This project is motivated by the need to design a lightweight, robust, and practical sign-based campus navigation system. By combining landmark-based feature extraction, temporal deep learning models, engineered geometric features, and a YOLO-based hand detection module, the study aims to develop and evaluate a structured recognition pipeline suitable for controlled real-world deployment.

1.2 EXISTING SYSTEM

Existing sign language recognition systems have seen significant advancements with the adoption of deep learning and computer vision techniques. Most current systems focus on large-scale sign

language translation tasks, where sequences of gestures are converted into text or speech. These approaches typically employ Convolutional Neural Networks (CNNs), Recurrent Neural Networks (RNNs), Long Short-Term Memory (LSTM) networks, and more recently, transformer-based architectures to capture spatial and temporal patterns in gesture sequences.

Many existing methods rely on raw image or video inputs for feature extraction. While these approaches are capable of learning complex visual representations, they often require large annotated datasets and high computational resources. Additionally, such pixel-based models are sensitive to background noise, lighting variations, and occlusions, which can reduce performance in real-world environments.

To address these challenges, some systems utilize landmark-based approaches, where hand keypoints are extracted using frameworks such as MediaPipe. These methods provide structured and low-dimensional representations of hand movements, improving computational efficiency and reducing sensitivity to background variations. However, landmark-only systems may struggle with accurate hand localization in cluttered scenes, leading to inconsistencies in feature extraction.

Recent studies have also explored detection-based approaches using object detection models such as YOLO to first localize the hand region before performing gesture recognition. While this improves spatial consistency and reduces noise, integrating detection and classification into a unified pipeline increases system complexity and computational overhead.

Despite these advancements, most existing systems are designed for continuous sign language translation or large vocabulary recognition, which may not be suitable for structured applications such as campus navigation. These applications require reliable recognition of a limited set of predefined gestures with high consistency rather than large-scale generalization.

Furthermore, many existing solutions do not explicitly address challenges such as small dataset sizes, class imbalance, and the need for modular system design that separates detection and classification stages. These limitations highlight the need for a lightweight, robust, and application-specific gesture recognition system tailored for structured environments.

The present study addresses these gaps by proposing a hybrid pipeline that combines landmark-based feature extraction, temporal deep learning models, engineered geometric features, and

YOLO-based hand detection to improve robustness and performance in a controlled campus navigation scenario.

1.3 PROBLEM STATEMENT

The problem addressed in this study is the development of a deep learning-based system capable of recognizing predefined campus block gestures from video input and mapping them to corresponding navigation instructions.

The task presents several challenges:

- Extracting meaningful hand features from raw video frames while minimizing background noise.
- Modeling temporal dependencies inherent in dynamic gestures.
- Handling limited data availability and class imbalance across gesture categories.
- Ensuring spatial consistency and robustness in feature extraction.
- Designing a modular pipeline that separates hand localization from gesture classification.

The problem is formulated as a closed-set multi-class gesture classification task involving nine predefined campus locations. Each input video sequence must be processed, segmented into fixed-length temporal windows, and classified into the correct gesture category. The final output must be deterministic and directly mapped to predefined navigation instructions.

1.4 OBJECTIVES

The objectives of this project are divided into modeling objectives and system-level objectives.

1.4.1 LANDMARK-BASED MODELING OBJECTIVES

- To extract 3D hand landmarks using MediaPipe (21 landmarks × x, y, z = 63 features per frame).
- To segment gesture videos into fixed-length temporal windows of 30 frames.
- To design and evaluate temporal deep learning architectures, including:
    - Baseline Bidirectional LSTM (BiLSTM),

  - BiLSTM with early stopping,
  - CNN–BiLSTM hybrid model for spatial–temporal learning.
- To enhance feature representation using:
  - Raw landmark coordinates,
  - Joint angle-based geometric features,
  - Velocity-based motion features.
- To address dataset imbalance using targeted landmark-level data augmentation.
- To evaluate model performance using:
  - Accuracy,
  - Precision,
  - Recall,
  - F1-score,
  - Confusion matrix,
  - Training and validation curves.

1.4.2 YOLO-BASED DETECTION AND HYBRID PIPELINE OBJECTIVES

- To design and train a YOLOv8-based hand detection model for robust hand localization.
- To prepare and annotate a detection dataset using structured frame extraction.
- To evaluate detection performance using:
  - Precision,
  - Recall,
  - mAP@0.5,
  - mAP@0.5–0.95.
- To implement an improved ROI extraction strategy including:
  - Largest bounding box selection,
  - Padding around detected hand,
  - Minimum size filtering,
  - Standardized resizing.
- To construct a hybrid pipeline: YOLO → ROI extraction → MediaPipe → CNN–BiLSTM.

- To compare landmark-only and YOLO-enhanced pipelines in terms of robustness and performance.

1.5 SCOPE OF THE PROJECT

This project focuses on the design, implementation, and evaluation of a sign-based campus navigation system under controlled conditions.

The system is limited to recognition of nine predefined campus block gestures captured from a single signer in an indoor environment. The task is treated as a closed-set classification problem rather than continuous sign language translation. Each gesture is segmented into fixed-length temporal sequences of 30 frames for model training and evaluation.

The study includes both landmark-based modeling and YOLO-based detection enhancement but does not extend to multi-signer generalization, large-scale datasets, real-time deployment optimization, or outdoor environment testing. Additionally, the system produces deterministic navigation outputs mapped directly from predicted classes and does not involve generative language models or conversational AI components.

The primary emphasis of the project is on comparative analysis of modeling strategies, feature engineering techniques, detection integration, and evaluation of system robustness within a structured academic framework. The system is evaluated in a semi-real-time environment however, real-time deployment in unconstrained environments may require further optimization and generalization.

The subsequent chapters present a detailed analysis of related work, dataset preparation, model development, experimental evaluation, and system-level insights derived from the study.

2. LITERATURE REVIEW

Sign Language Processing (SLP) has gained significant attention in recent years due to advances in deep learning, computer vision, and natural language processing. SLP broadly includes three major tasks: Sign Language Recognition (SLR), Sign Language Translation (SLT), and Sign Language Generation (SLG). A comprehensive survey by Tan et al. (2024) [1] provides an extensive review of deep learning-based approaches for sign language processing and highlights key challenges such as multimodal representation, dataset variability, gloss dependency, and inconsistencies in evaluation metrics. The survey emphasizes that while recognition performance has improved substantially, translation systems still struggle with generalization and robustness due to limited datasets and complex linguistic structures.

Sign language recognition systems can be categorized into isolated and continuous recognition tasks. Liang et al. (2023) [2] distinguish between sign-to-gloss and sign-to-text frameworks and describe four major paradigms: Sign2Gloss2Text, Sign2Text, Sign2(Gloss+Text), and Gloss2Text. These frameworks clarify the conceptual difference between recognizing sign glosses and translating them into spoken language. While translation models focus on sentence-level generation, many practical systems, particularly in structured environments, operate under closed-set recognition settings where predefined gesture classes are identified. The present study aligns with this latter paradigm.

Recent research has demonstrated the effectiveness of landmark-based approaches for gesture recognition. Instead of learning directly from raw image pixels, these systems extract keypoints representing hand or body joints and use them as structured input features. Maia et al. (2025) [3] propose a transformer-based architecture that leverages body keypoints extracted using MediaPipe and show that keypoint embeddings can achieve competitive translation performance compared to pixel-based approaches. Their work highlights the importance of structured representations and augmentation strategies in improving robustness. Similarly, Alsharif et al. (2025) [4] integrate YOLO-based gesture detection with MediaPipe keypoint extraction to build a real-time American Sign Language interpretation system, achieving high detection precision and real-time inference speed. Ravikiran (2025) [5] further demonstrates that LSTM-based temporal modeling combined with MediaPipe landmarks can effectively recognize dynamic gestures in real-time environments.

Temporal modeling is critical in dynamic gesture recognition, as signs are inherently spatiotemporal. Pandey et al. (2025) [6] compare CNN, LSTM, and transformer-based architectures for vision-based sign language translation and report that models incorporating temporal dependencies significantly outperform static classifiers. Camgoz et al. (2020) [7] introduce transformer-based joint recognition and translation models and demonstrate that gloss-level intermediate representations substantially improve translation performance. Their work highlights that modeling sequential dependencies across frames is essential for capturing dynamic hand movements and contextual transitions. These findings justify the use of recurrent architectures such as BiLSTM and hybrid CNN–BiLSTM models in structured gesture recognition tasks.

In addition to spatial–temporal modeling, recent studies explore multimodal fusion and occlusion-resilient systems. Damdoo and Kumar (2025) [8] propose a three-stream multimodal transformer that integrates visual, gesture, and emotional cues to improve robustness under occlusion conditions. Their work demonstrates that combining multiple modalities enhances translation accuracy, particularly in continuous sign language recognition tasks. The importance of large-scale datasets for training such models is highlighted in the How2Sign dataset introduced by Duarte et al. [9], which provides over 80 hours of multimodal American Sign Language data. However, large transformer-based models often require extensive computational resources and annotated datasets, making them less practical for small-scale structured applications.

Object detection has also emerged as a critical component in robust gesture recognition systems. Alsharif et al. [4] show that integrating YOLO with keypoint tracking improves hand localization and reduces background noise. Detection-based preprocessing ensures that feature extraction focuses only on relevant regions of interest (ROI), thereby improving classification reliability. Modern YOLO implementations, supported by Ultralytics documentation, provide efficient training and inference pipelines, while tools such as Roboflow facilitate structured dataset preparation and annotation in YOLO format. These developments motivate the integration of a YOLO-based hand detection module prior to landmark extraction in the present study.

Despite substantial progress in SLR and SLT, several research gaps remain. Most state-of-the-art works focus on continuous sentence-level translation tasks requiring large-scale datasets [1], [2],

[9]. Limited research systematically compares landmark-only pipelines with detection-enhanced pipelines under structured conditions. Furthermore, the impact of engineered features such as joint angles and velocity-based motion descriptors has not been extensively analyzed in small closed-set datasets. Dataset imbalance and its effect on temporal modeling performance are also underexplored in structured gesture recognition systems.

The present study addresses these gaps by conducting a structured comparative evaluation of landmark-based temporal models, incorporating geometric and motion-based feature engineering, and introducing a YOLO-enhanced detection-classification pipeline. Unlike large-scale translation systems, this work focuses on a closed-set campus navigation scenario, emphasizing robustness, modular design, and comparative analysis within a limited but controlled dataset.

3. SYSTEM DESIGN

3.1 SYSTEM OVERVIEW

The proposed system is a deep learning-based sign language recognition framework designed to assist in structured campus navigation. The system interprets predefined hand gestures corresponding to campus locations and maps them to navigation outputs in a deterministic manner.

Unlike traditional sign language systems that focus on large vocabulary translation, this system operates in a **closed-set environment** with a limited number of gesture classes. This allows for a more controlled, efficient, and application-specific design.

The system integrates multiple stages of processing, including video acquisition, frame extraction, hand detection, landmark extraction, feature engineering, and temporal modeling. These components are organized into a modular pipeline to ensure scalability, maintainability, and robustness.

Two system configurations are explored:

3.1.1 LANDMARK-BASED PIPELINE: Direct extraction of hand landmarks from full-frame input using MediaPipe.

3.1.2 DETECTION-ENHANCED PIPELINE: Integration of YOLO-based hand detection to improve spatial localization before landmark extraction.

The hybrid design allows comparative evaluation of both approaches and enables improved performance under varying conditions. The overall interaction between the user and the system is illustrated in Fig 3.1.

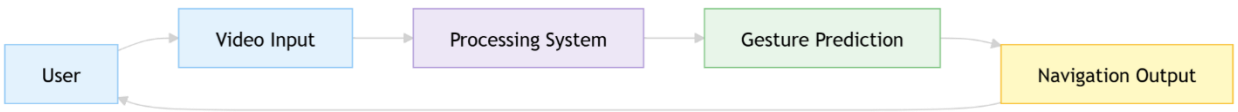


Fig 3.1. Overall System Interaction Diagram

3.2 SYSTEM ARCHITECTURE

The system architecture defines the complete processing pipeline that transforms raw video input into a classified gesture output. The architecture follows a **sequential and modular design**, where each stage performs a well-defined operation on the input data.

The pipeline can be summarized as:

Input Video → Frame Extraction → Hand Detection → ROI Extraction → Landmark Extraction → Feature Engineering → Temporal Modeling → Classification → Navigation Output

3.2.1 ARCHITECTURAL CHARACTERISTICS

- **Modular Design**: Each component operates independently, allowing easy modification and replacement.
- **Pipeline Processing**: Data flows sequentially through multiple transformation stages.
- **Hybrid Modeling**: Combines spatial detection (YOLO), geometric representation (MediaPipe), and temporal modeling (BiLSTM/CNN–BiLSTM).
- **Scalability**: New modules (e.g., multi-hand detection or gesture expansion) can be added without redesigning the system.

Table 3.1. Data Representation Across Stages

Stage	**Representation**
Input	RGB Video
Frame Extraction	Image frames
Detection	Bounding boxes
Landmark Extraction	63-Dimension feature vector
Feature Engineering	Enhanced feature vector

Temporal Modeling	Sequence (30 × features)
Output	Class probabilities

The complete processing pipeline of the proposed system is shown in Fig 3.2.

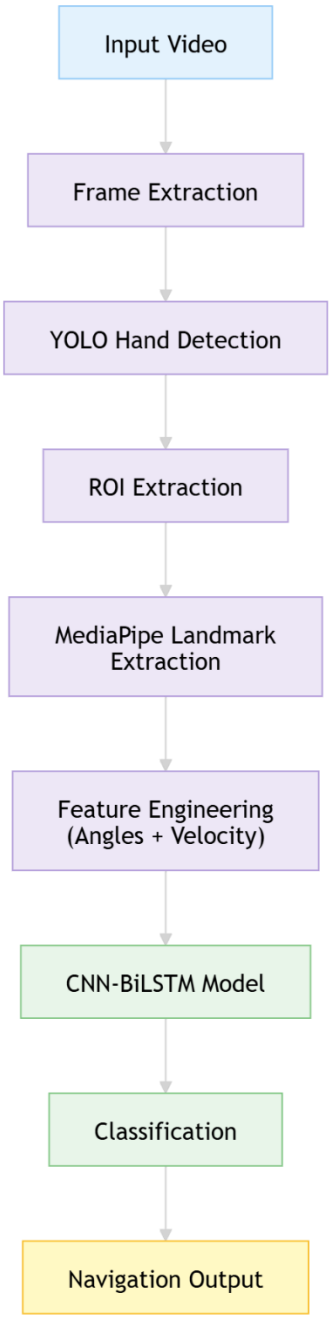


Fig 3.2. System Architecture of Proposed Model

3.3 MODULE DESCRIPTION

The system is composed of several functional modules, each responsible for a specific processing task. The modular breakdown of the system components is depicted in Fig 3.3.

3.3.1 INPUT MODULE

This module captures gesture input in the form of video streams. The system supports both real-time webcam input and pre-recorded video files.

Key functions:

- Video acquisition
- Input format standardization
- Frame buffering

3.3.2 PREPROCESSING MODULE

The preprocessing stage prepares raw input data for further analysis.

Operations include:

- Frame extraction from video
- Resizing and normalization
- Sequence segmentation (fixed length: 30 frames)
- Handling missing or redundant frames

This ensures consistent input structure across all samples.

3.3.3 DETECTION MODULE (YOLO)

This module performs hand localization using YOLOv8.

Key functionalities:

- Detects hand bounding boxes
- Outputs confidence scores

- Filters low-confidence detections

Significance:

- Reduces background noise
- Improves spatial consistency
- Enhances robustness in real-world conditions

3.3.4 ROI EXTRACTION MODULE

After detection, the Region of Interest (ROI) is extracted.

Processing steps:

- Select highest-confidence detection
- Apply padding to bounding box
- Resize ROI to standard dimensions
- Handle multiple detections

This ensures that only relevant hand regions are processed further.

3.3.5 FEATURE EXTRACTION MODULE (MEDIAPIPE)

This module extracts structured hand landmarks.

Details:

- 21 keypoints per hand
- Each point: (x, y, z)
- Output: 63-dimensional vector per frame

Advantages:

- Reduced dimensionality
- Robust to lighting variations
- Focuses on geometric structure

3.3.6 FEATURE ENGINEERING MODULE

This stage enhances raw landmark features.

Techniques used:

- Joint angle computation
- Velocity (temporal difference) calculation

Benefits:

- Captures geometric relationships
- Encodes motion dynamics
- Improves class separability

3.3.7 TEMPORAL MODELING MODULE

This module captures time-dependent patterns in gesture sequences.

Models used:

- Bidirectional LSTM
- CNN–BiLSTM hybrid

Functionality:

- Processes sequential input
- Learns temporal dependencies
- Captures motion patterns

3.3.8 CLASSIFICATION MODULE

The classification stage maps learned features to gesture labels.

Details:

- Multi-class classification (9 classes)

- Softmax activation
- Output: probability distribution

3.3.9 OUTPUT MODULE

This module generates the final system output.

Functions:

- Displays predicted gesture
- Maps to navigation instructions
- Provides user feedback



Fig 3.3. Modular System Design

3.4 SYSTEM WORKFLOW

The system workflow describes the step-by-step execution process. The step-by-step execution workflow of the system is illustrated in Fig 3.4.

Workflow Steps:

1. Capture video input from user
2. Extract frames from input stream
3. Detect presence of hand using YOLO
4. If hand is detected:
   - Extract ROI
   - Process frame

   Else:
   - Skip frame
5. Extract hand landmarks

6. Apply feature engineering
7. Pass sequence to temporal model
8. Predict gesture class
9. Display output

Key Characteristics:

- Iterative processing across frames
- Conditional decision-making (hand detection)
- Sequential transformation of data

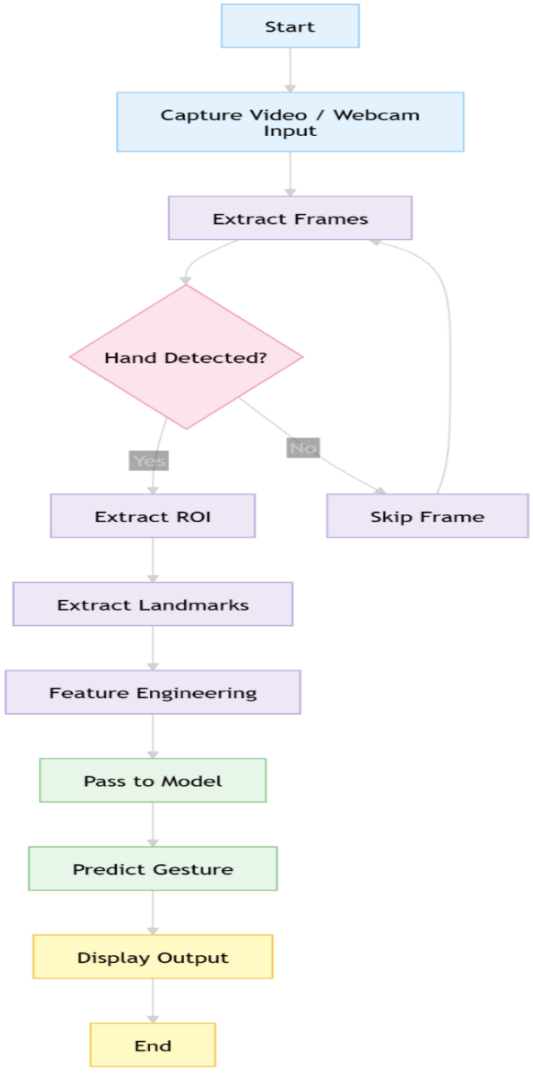


Fig 3.4. System Workflow

3.5 DESIGN CONSIDERATIONS AND TRADE-OFFS

The system design balances multiple competing factors:

3.5.1 ACCURACY VS EFFICIENCY

- Detection improves accuracy
- Increases computation time

3.5.2 MODEL COMPLEXITY VS DATASET SIZE

- Complex models risk overfitting
- Simpler models generalize better

3.5.3 SPATIAL VS TEMPORAL MODELING

- Landmark features capture spatial structure
- LSTM captures temporal dynamics

3.5.4 ROBUSTNESS VS REAL-TIME PERFORMANCE

- Detection increases robustness
- May reduce real-time speed

3.6 ADVANTAGES OF THE PROPOSED SYSTEM

- Modular and scalable architecture
- Efficient due to landmark representation
- Improved robustness with detection
- Suitable for structured applications
- Supports real-time implementation

3.7 LIMITATIONS OF SYSTEM DESIGN

- Limited dataset size
- Single-user dependency

- Reduced generalization capability
- Computational overhead due to detection
- Sensitivity to extreme lighting conditions

Despite these limitations, the proposed system provides a robust and scalable framework for structured gesture-based navigation and serves as a strong foundation for future real-world deployment.

4. DATASET AND PREPROCESSING

This chapter describes the dataset collection process, preprocessing strategies, feature extraction methods, and preparation steps for both the landmark-based classification pipeline and the YOLO-based detection-enhanced pipeline.

4.1 DATASET COLLECTION AND DESCRIPTION

The dataset used in this study was collected specifically for a structured sign-based campus navigation system. The objective was to recognize a predefined set of nine campus block gestures, each corresponding to a distinct navigation location. Therefore, the problem was formulated as a closed-set multi-class gesture classification task.

A total of 30 videos were recorded in a controlled indoor environment using a standard RGB camera. The recordings were captured under stable lighting conditions to minimize environmental variability. All gestures were performed by a single signer to maintain consistency in execution style, speed, and articulation. This controlled setup helped reduce inter-signer variability and allowed the study to focus on model performance rather than signer generalization.

Each video contains a dynamic gesture sequence rather than a static pose. Since gestures evolve over time, the dataset was designed for sequence-based modeling instead of frame-level classification. The videos vary slightly in duration depending on the speed of execution but follow a consistent gesture structure.

Before further processing, all videos were manually reviewed to ensure quality. The validation process ensured that:

- The hand region was clearly visible throughout the gesture,
- There was minimal motion blur,
- The gesture was fully completed within the recording,
- Severe occlusions or truncations were absent.

Only videos satisfying these criteria were retained for modeling.

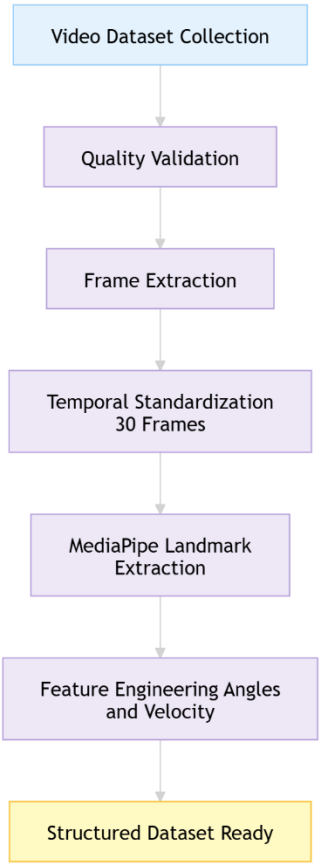


Fig 4.1. Dataset Collection and Preprocessing Pipeline

4.2 FRAME EXTRACTION AND TEMPORAL STANDARDIZATION

To convert raw video data into structured model inputs, frame extraction was performed. Each video was decomposed into individual frames using uniform sampling.

Since deep learning models require fixed input dimensions, each gesture sample was standardized to a sequence length of 30 frames. For the landmark-based pipeline, each gesture sample was standardized to a sequence length of 30 frames. However, in the YOLO-based pipeline, sequences retained their natural temporal length (ranging from 20 to 35 frames) and were later padded to a maximum sequence length during temporal modeling. This temporal normalization ensures that all samples have consistent shape regardless of original video duration.

If a video contained more than 30 frames, uniform downsampling was applied. Frames without detected hand landmarks were discarded. Sequences with excessive missing frames were excluded

to maintain data quality and temporal consistency. This fixed-length segmentation was essential for training recurrent architectures such as BiLSTM and CNN–BiLSTM models.

After this step, each gesture sample was represented as a structured temporal sequence ready for feature extraction.

4.3 LANDMARK EXTRACTION AND FEATURE REPRESENTATION

For the primary modeling pipeline, hand landmarks were extracted using MediaPipe Hands. MediaPipe provides 21 keypoints per detected hand, with each keypoint represented by three coordinates (x, y, z). Thus, each frame was converted into a 63-dimensional feature vector.

Landmark-based representation offers several advantages:

- Significant dimensionality reduction compared to raw image pixels,
- Structured geometric encoding of hand pose,
- Reduced sensitivity to background noise,
- Improved computational efficiency.

In cases where multiple hands were detected, the dominant hand was selected based on detection confidence and bounding box size. Frames with missing detections were handled through interpolation to preserve temporal continuity.

After extraction, each gesture sample was represented as a tensor of size: 30×63 representing 30 frames and 63 spatial features per frame.

To further enhance discriminative capability, feature engineering techniques were applied. Two additional feature types were computed:

4.3.1 JOINT ANGLE FEATURES:

Angles were calculated between connected landmark triplets to capture geometric relationships independent of absolute position. These angle-based features improve robustness to scale and minor spatial shifts.

4.3.2 VELOCITY FEATURES:

Frame-to-frame differences in landmark coordinates were computed to capture motion dynamics. Velocity-based descriptors help distinguish gestures with similar static poses but different temporal patterns.

The final feature representation combined raw landmark coordinates with engineered geometric and motion-based features, resulting in a richer temporal representation for model training.

4.4 DATASET SPLITTING AND IMBALANCE HANDLING

The dataset was divided into training, validation, and testing subsets using stratified splitting to preserve class distribution across subsets. To prevent data leakage, a video-aware splitting strategy was implemented. All frames belonging to a single video were assigned to the same subset (training, validation, or testing). This ensured that the model did not learn from similar frames across different splits, preserving the integrity of evaluation.

Initial class distribution analysis revealed imbalance among gesture categories. Since imbalanced datasets can bias deep learning models toward majority classes, targeted augmentation was applied at the landmark level.

Augmentation strategies included:

- Small Gaussian noise injection in landmark coordinates,
- Controlled spatial perturbation,
- Temporal jittering of sequences.

Augmentation was applied primarily to minority classes until a minimum sample threshold was achieved. This balancing strategy improved macro-level evaluation metrics and reduced prediction bias.

4.5 YOLO-BASED DETECTION DATASET PREPARATION

In addition to the landmark-based pipeline, a detection-enhanced pipeline was developed using YOLO.

For this purpose, extracted video frames were uploaded to Roboflow for annotation in YOLO format [12]. Bounding boxes were manually drawn around the hand region in each frame. The annotated dataset was automatically split into training, validation, and testing subsets.

YOLOv8 was trained using the Ultralytics implementation [11], based on the original YOLO object detection framework [10]. The objective of this model was to localize hand regions accurately within each frame before landmark extraction.

The detection model was evaluated using standard object detection metrics including:

- Precision
- Recall
- mAP@0.5
- mAP@0.5–0.95

This detection step improved robustness by isolating the hand region and minimizing the influence of background noise.

4.6 ROI EXTRACTION AND HYBRID PIPELINE

After training, YOLO was used to generate bounding boxes for hand localization in each frame. A structured ROI extraction strategy was implemented:

- The largest bounding box was selected when multiple detections occurred,
- Padding was applied to preserve contextual spatial information,
- Low-confidence detections were filtered,
- Cropped regions were resized to a standardized resolution.

The extracted ROI was then passed to the MediaPipe landmark extraction module. This created a modular hybrid pipeline: YOLO → ROI Extraction → MediaPipe → Temporal Model

This design separates detection from classification, improving spatial consistency and robustness in varying conditions. After ROI extraction, the dataset size was reduced from 834 frames to 412 frames, as low-quality or irrelevant frames were filtered out. This improved dataset quality while reducing noise.

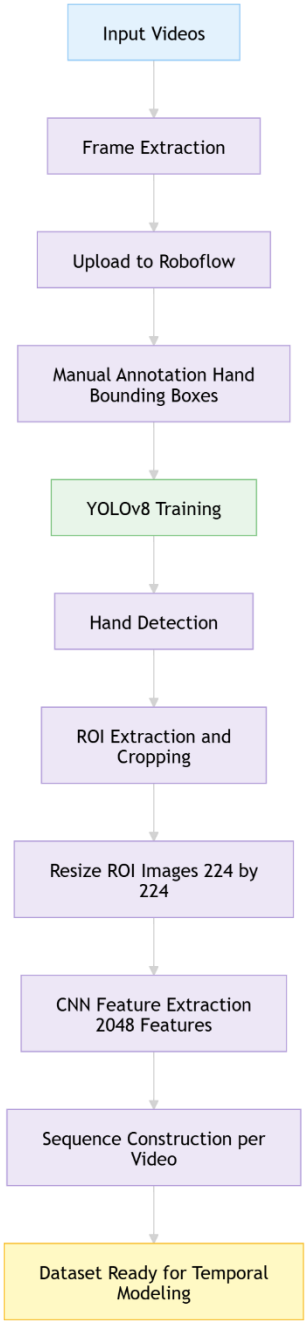


Fig 4.2. YOLO-Based Data Preparation Pipeline

4.7 CNN-BASED FEATURE EXTRACTION (YOLO PIPELINE)

In the YOLO-based pipeline, instead of directly extracting landmarks, a CNN-based feature extraction approach was employed. The cropped ROI images were passed through a pretrained convolutional neural network (CNN) used as a feature extractor.

Each ROI image (224 × 224) was converted into a 2048-dimensional feature vector representing spatial hand characteristics such as shape, orientation, and finger configuration.

These features provided a rich spatial representation compared to raw pixel data and were used as input for temporal modeling using BiLSTM.

The extracted features were stored and organized based on video sequences, ensuring compatibility with sequence-based deep learning models.

5. LANDMARK-BASED TEMPORAL MODELING

This chapter presents the first phase of the proposed system, focusing on gesture recognition using structured hand landmark representations and temporal deep learning models. The primary objective of this phase was to evaluate how effectively dynamic gestures can be modeled using geometric hand keypoints without relying on explicit object detection.

Unlike pixel-based approaches, which operate directly on raw image data, this method utilizes low-dimensional landmark representations extracted using MediaPipe. This significantly reduces computational complexity while preserving essential geometric and motion-related information. The study systematically explores baseline temporal modeling, training strategies, feature engineering, class imbalance handling, and hybrid spatial–temporal architectures.

5.1 PROBLEM FORMULATION AND INPUT REPRESENTATION

The gesture recognition problem is formulated as a supervised multi-class classification task involving nine predefined campus gesture classes. Since gestures evolve over time, the problem requires modeling both spatial and temporal dependencies.

Each gesture sample is represented as a sequence of 30 frames. For each frame, 21 hand landmarks are extracted, with each landmark consisting of three coordinates (x, y, z), resulting in a 63-dimensional feature vector per frame.

Thus, each input sample is represented as: $\mathbf{X \in R^{30\times 63}}$

where:

- 30 represents the temporal dimension (frames),
- 63 represents spatial features per frame.

The classification objective is to learn a mapping:

$$\mathbf{f: R^{30\times 63} \rightarrow R^{9}}$$

where the output corresponds to probability scores across nine gesture classes.

5.2 BASELINE BIDIRECTIONAL LSTM ARCHITECTURE

To model temporal dependencies, a Bidirectional Long Short-Term Memory (BiLSTM) network was implemented as the baseline model. Unlike traditional feedforward networks, LSTMs can retain information across time steps, making them suitable for sequential data such as gestures.

The bidirectional configuration processes sequences in both forward and backward directions, enabling the model to capture contextual dependencies across the entire gesture sequence.

The baseline architecture consists of:

- One Bidirectional LSTM layer (64 units)
- Dropout layer for regularization
- Fully connected dense layer (ReLU activation)
- Softmax output layer (9 classes)

The model was trained using categorical cross-entropy loss and the Adam optimizer.

Training Configuration

The baseline model was trained under controlled conditions with:

- 30 training epochs
- Batch size of 16
- No data augmentation
- No early stopping

This configuration was intentionally chosen to establish a reference point for evaluating improvements.

5.3 TRAINING STRATEGY OPTIMIZATION

Initial experiments revealed signs of overfitting, where training performance improved while validation performance stagnated. This behavior is common in small datasets.

To address this, early stopping was introduced based on validation loss monitoring. The model training was halted when validation performance stopped improving, and the best-performing weights were restored.

This strategy improved model generalization and reduced unnecessary training iterations. However, analysis revealed that performance limitations were not solely due to overfitting but were significantly influenced by dataset imbalance.

5.4 FEATURE ENGINEERING FOR ENHANCED REPRESENTATION

Raw landmark coordinates capture positional information but do not explicitly encode geometric relationships or motion characteristics. To enhance representation, feature engineering techniques were introduced.

5.4.1 ANGLE-BASED FEATURES

Joint angles were computed using triplets of connected landmarks. These features represent intrinsic hand geometry and are invariant to translation and scale.

Angle features:

- Capture finger articulation
- Improve differentiation between similar gestures
- Provide biomechanically meaningful representation

This increased the feature dimension from 63 to 84 per frame.

5.4.2 VELOCITY-BASED FEATURES

To capture motion dynamics, velocity features were computed as frame-to-frame differences:

$$v_t = x_t - x_{t-1}$$

Velocity features encode:

- Direction of movement

- Speed of motion
- Temporal transitions

When combined with static features, the feature dimension increased to 168 per frame.

However, experimental observations indicated that higher feature dimensionality introduced noise and reduced model stability in small datasets.

5.5 HANDLING CLASS IMBALANCE

Dataset analysis revealed uneven distribution across gesture classes, leading to biased predictions toward majority classes.

To address this, targeted landmark-level augmentation was applied only to underrepresented classes. Augmentation techniques included:

- Controlled Gaussian noise
- Spatial perturbations
- Temporal jitter

This ensured balanced class representation while preserving gesture semantics. Unlike uniform augmentation, this targeted approach avoided distorting well-represented classes and improved fairness across categories.

5.6 CNN–BILSTM HYBRID ARCHITECTURE

While BiLSTM captures temporal dependencies, it assumes optimal spatial representation. To enhance spatial feature extraction, a hybrid CNN–BiLSTM architecture was introduced.

5.6.1 SPATIAL MODELING

1D convolutional layers were applied across feature dimensions to capture:

- Local landmark relationships
- Finger group interactions
- Spatial configurations

5.6.2 TEMPORAL MODELING

The extracted spatial features were passed to BiLSTM layers to model temporal evolution.

This hybrid approach enabled:

- Intra-frame spatial learning (CNN)
- Inter-frame temporal modeling (BiLSTM)

5.7 EXPERIMENTAL EXPLORATION AND MODEL VARIANTS

Multiple configurations were systematically evaluated to understand the impact of different design choices:

- Baseline BiLSTM
- BiLSTM with early stopping
- BiLSTM with targeted augmentation
- CNN–BiLSTM (landmarks only)
- CNN–BiLSTM with angle features
- CNN–BiLSTM with angle + velocity features
- Deeper CNN–BiLSTM architecture

Each configuration contributed to understanding trade-offs between feature complexity, model capacity, and dataset size.

5.8 OBSERVATIONS AND INSIGHTS

Several key insights emerged from the experimental analysis:

- Temporal modeling significantly improves gesture recognition over static approaches
- Dataset imbalance is the primary limiting factor in performance
- Targeted augmentation improves minority class representation
- Angle-based features enhance geometric understanding and stability
- Velocity features introduce noise in small datasets
- Increasing model depth leads to overfitting without sufficient data

- Feature quality is more important than model complexity

These findings highlight that careful feature design and data balancing are more effective than increasing architectural complexity.

5.9 FINAL MODEL SELECTION

Based on experimental evaluation, the CNN–BiLSTM model with angle-based features was selected as the final landmark-based model.

This configuration provided:

- Balanced performance across classes
- Improved stability and generalization
- Optimal trade-off between complexity and accuracy

This model serves as the baseline for comparison with the YOLO-enhanced pipeline presented in the next chapter.

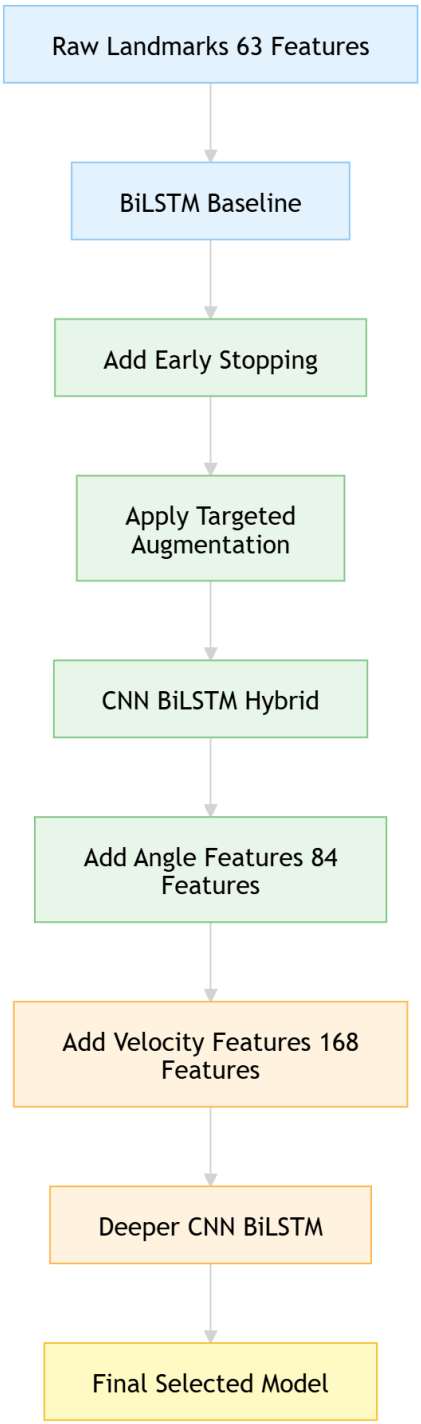


Fig 5.1. Evolution of Landmark-Based Models and Feature Enhancements

6. YOLO-BASED HAND DETECTION AND HYBRID PIPELINE

This chapter presents the second phase of the proposed system, which introduces a detection-enhanced pipeline to improve robustness and spatial consistency in gesture recognition. While the landmark-based approach described in Chapter V relies on direct landmark extraction from full-frame images, such methods may suffer from background noise, scale variation, and inconsistent hand positioning.

To address these limitations, a YOLO-based hand detection module was integrated as a preprocessing stage. This module isolates the hand region before feature extraction, enabling more reliable and consistent downstream modeling.

6.1 RATIONALE FOR DETECTION-BASED ENHANCEMENT

In the landmark-only pipeline, MediaPipe operates directly on full-frame images. Although efficient, it may struggle under conditions such as:

- Background clutter
- Varying hand positions
- Scale inconsistencies
- Partial occlusions
- Frame-to-frame positional shifts

To overcome these limitations, an object detection stage was introduced to first localize the hand region. By extracting a Region of Interest (ROI), the system ensures that only relevant hand pixels are processed further.

YOLOv8 was selected due to its real-time performance and strong accuracy-speed trade-off. In this study, a lightweight variant (YOLOv8n) was used to maintain computational efficiency.

Importantly, YOLO performs **only localization**, not gesture recognition. Its role is to act as a preprocessing gatekeeper that isolates the hand region for subsequent analysis.

6.2 DETECTION DATASET CONSTRUCTION

The detection dataset was derived from the original gesture videos.

6.2.1 FRAME EXTRACTION STRATEGY

Each video was decomposed into individual frames, as YOLO operates on image inputs rather than video sequences. Frames were sampled uniformly across gesture duration to capture:

- Start, mid-motion, and end phases
- Variations in hand pose
- Balanced temporal representation

This process resulted in approximately **834 frames**, forming the dataset used for detection training.

6.3 DATASET ANNOTATION AND MANAGEMENT USING ROBOFLOW

Roboflow was used as a dataset management and annotation platform.

6.3.1 ANNOTATION DESIGN

- Single class: **"hand"**
- Gesture labels were not used at this stage
- Bounding boxes were drawn tightly around the hand region

This design choice ensured that YOLO focuses purely on localization.

6.3.2 PREPROCESSING AND AUGMENTATION

Roboflow preprocessing included:

- Auto-orientation
- Resizing to YOLO-compatible resolution (640×640)
- Moderate augmentation (flip, brightness, contrast, scaling)

These steps improved generalization while preserving hand structure.

6.3.3 DATASET SPLITTING

Initial dataset split:

- Training: 584 images
- Validation: 167 images
- Test: 83 images

6.4 YOLOV8 MODEL TRAINING

The YOLOv8 model was trained using transfer learning with pretrained weights.

6.4.1 MODEL CONFIGURATION

- Model: YOLOv8n
- Input size: 640×640
- Single class: hand

6.4.2 EVALUATION METRICS

- Precision
- Recall
- mAP@0.5
- mAP@0.5–0.95

These metrics evaluate detection accuracy and bounding box quality.

The trained model demonstrated strong localization capability across different gesture variations and backgrounds.

6.5 ROI EXTRACTION AND DATASET REFINEMENT

After detection, bounding boxes were used to extract hand regions.

6.5.1 ROI EXTRACTION STRATEGY

1. The following rules were applied:

2. Select highest-confidence detection
3. Choose largest bounding box (if multiple)
4. Add ~15% padding
5. Remove small regions (< 80×80)
6. Resize ROI to 224×224

This ensured preservation of full hand structure while standardizing inputs.

6.5.2 IMPACT OF ROI EXTRACTION

- Total frames reduced from **834 → 412**
- Removal of blurred and transition frames
- Improved dataset quality and consistency

This step significantly reduced noise and enhanced spatial alignment.

6.6 UNIFIED DATASET AND VIDEO-AWARE SPLITTING

A critical limitation of frame-level splitting is **data leakage**, where frames from the same video appear in different splits.

To address this, a **video-aware dataset split** was implemented:

- Training: 19 videos
- Validation: 5 videos
- Test: 6 videos

All frames belonging to a single video were kept in the same subset.

Metadata fields used:

- class label
- video_id
- frame_index
- image_path

This ensured proper temporal integrity and realistic evaluation.

6.7 CNN-BASED FEATURE EXTRACTION

Instead of directly using ROI images for classification, a pretrained CNN was used as a **feature extractor**.

6.7.1 FEATURE EXTRACTION PROCESS

- Input: ROI images (224×224)
- Output: 2048-dimensional feature vector per frame

The CNN captures:

- Hand shape
- Finger configuration
- Spatial posture

This converts high-dimensional image data into compact feature representations suitable for temporal modeling.

6.8 TEMPORAL SEQUENCE CONSTRUCTION

Since gestures are dynamic, individual frames are insufficient.

6.8.1 SEQUENCE FORMATION

- Frames grouped by video_id
- Ordered by frame_index
- Features stacked into sequences (T × 2048)
- Sequences padded to equal length (max = 31 frames)

Final dataset shape:
(30 videos × 31 frames × 2048 features)

This representation enables effective temporal modeling using BiLSTM.

6.9 CLASS IMBALANCE HANDLING

Dataset imbalance persisted even after ROI extraction.

To address this, **class-weighted loss** was used:

- Minority classes assigned higher weights
- Majority classes assigned lower weights

This ensures balanced learning and prevents bias toward frequent classes.

6.10 INTEGRATION INTO HYBRID PIPELINE

The final hybrid pipeline operates as:

YOLO → ROI Extraction → CNN Feature Extraction → Sequence Formation → BiLSTM Classification

This modular design separates:

- Detection (YOLO)
- Spatial representation (CNN)
- Temporal modeling (BiLSTM)

This improves robustness and allows independent optimization of each stage.

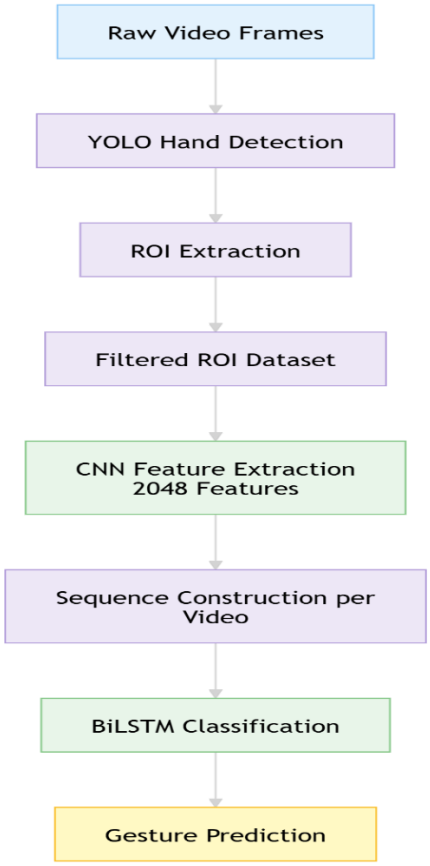


Fig 6.1. Transformation Pipeline from Raw Frames to Gesture Prediction

6.11 DESIGN CONSIDERATIONS AND TRADE-OFFS

6.11.1 ADVANTAGES

- Improved spatial consistency
- Reduced background noise
- Better feature representation
- Modular pipeline design

6.11.2 TRADE-OFFS

- Increased computational complexity
- Additional preprocessing steps
- Larger pipeline structure

6.12 KEY INSIGHTS

- Detection significantly improves input quality
- ROI extraction removes irrelevant information
- CNN features provide strong spatial encoding
- Video-aware splitting prevents data leakage
- Feature-based pipelines outperform raw pixel inputs in structured tasks

7. RESULTS AND COMPARATIVE ANALYSIS

This chapter presents the experimental evaluation of both the landmark-based temporal modeling pipeline and the YOLO-enhanced hybrid pipeline. The objective of this evaluation is to systematically analyze classification performance, detection robustness, model generalization behavior, and the comparative effectiveness of architectural modifications.

All experiments were conducted under consistent hardware and preprocessing conditions to ensure fairness in evaluation.

7.1 LANDMARK-BASED MODEL RESULTS

7.1.1 BASELINE BILSTM PERFORMANCE

The baseline Bidirectional LSTM model was trained using only raw landmark features without early stopping or data augmentation.

Test Accuracy: ~32.6%

This low performance indicates that the model struggled to generalize across gesture classes. Examination of the confusion matrix revealed strong bias toward majority classes, particularly RDBlock and Examination, while several block classes were not predicted at all.

This confirms two major issues:

- Severe class imbalance
- Insufficient spatial representation using raw coordinates alone

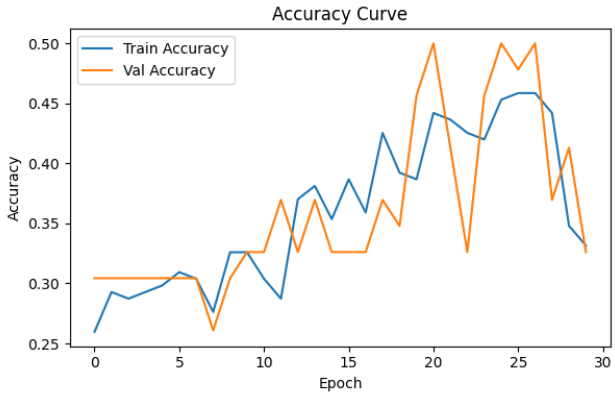


Fig 7.1. Training and Validation Accuracy Curve (Baseline BiLSTM)

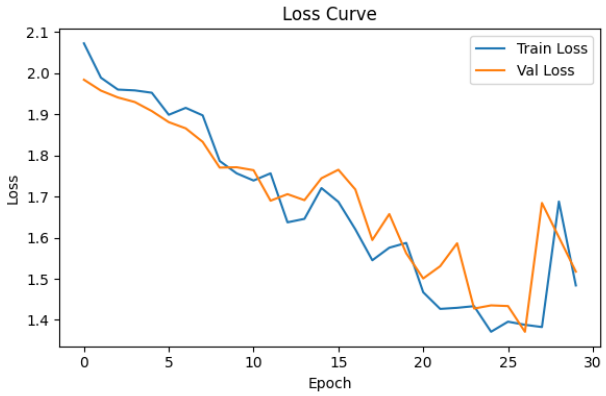


Fig 7.2. Training and Validation Loss Curve (Baseline BiLSTM)

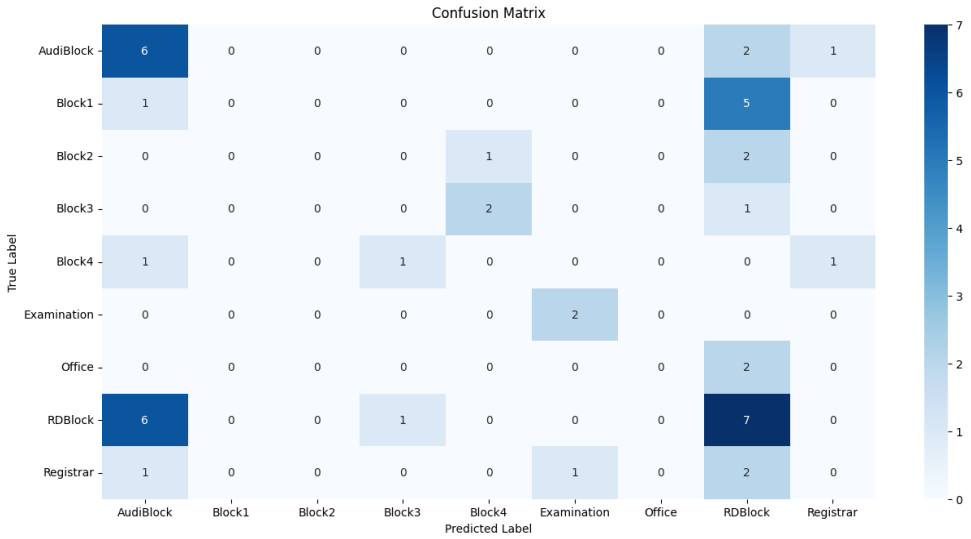


Fig 7.3. Confusion Matrix of Baseline BiLSTM

The training curve likely shows increasing training accuracy but stagnant validation accuracy. This indicates overfitting due to limited dataset size and class imbalance.

7.1.2 BILSTM WITH EARLY STOPPING

Introducing early stopping improved generalization.

Test Accuracy: ~52.2%

This represents an approximate 20% absolute improvement over the baseline.

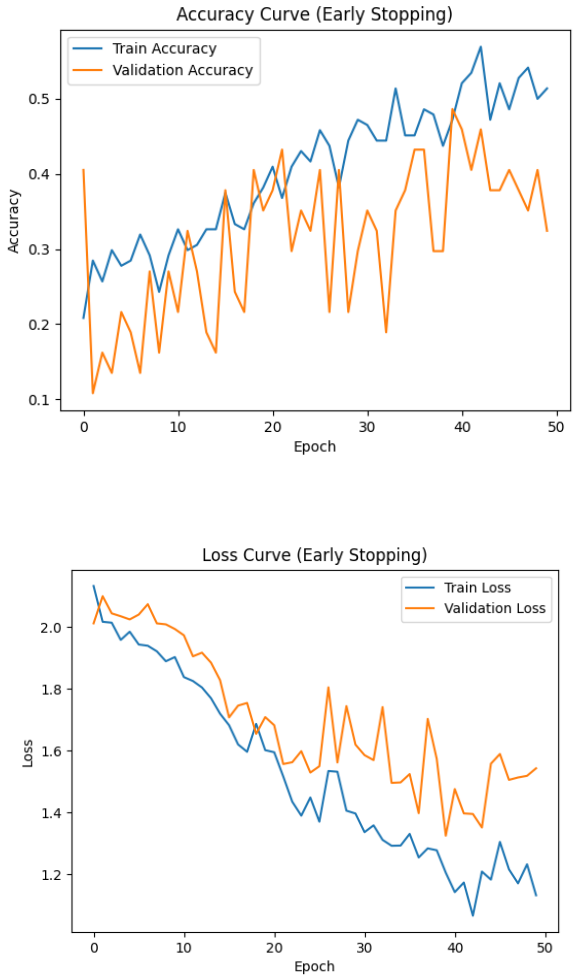


Fig 7.4. Training and Validation Curves with Early Stopping

Early stopping prevented the model from memorizing majority-class patterns. Validation loss stabilized earlier, and minority class recall improved slightly. However, imbalance still limited performance ceiling.

7.1.3 BILSTM WITH TARGETED DATA AUGMENTATION

A minimum threshold of 40 samples per class was enforced. Classes below this threshold were augmented using landmark-level perturbations, while majority classes such as RDBlock and AudiBlock were left unchanged. This resulted in a balanced dataset of 394 samples, ensuring fair representation across all gesture classes. After applying landmark-level augmentation to minority classes:

Test Accuracy: 67.09%

Macro F1-score: 0.68

This experiment demonstrates that **data balancing had the most significant impact** on classification performance.

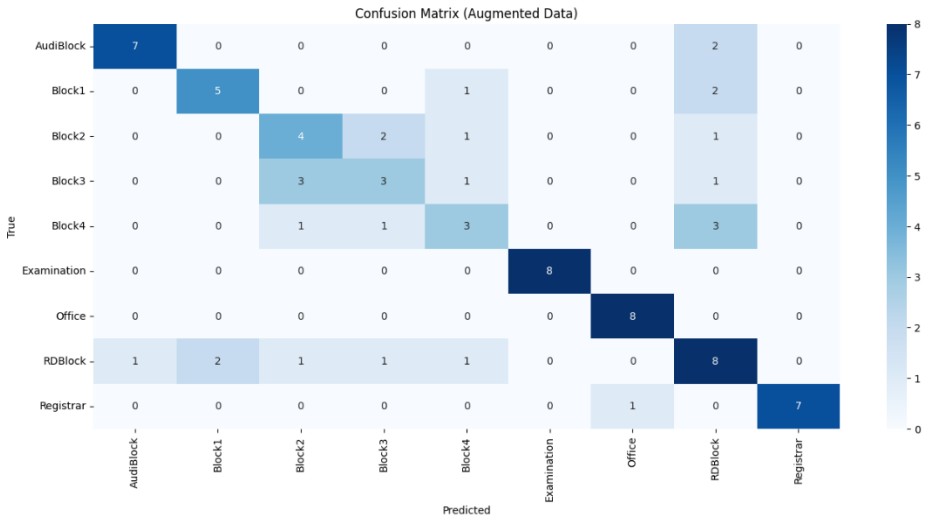


Fig 7.5. Confusion Matrix after Targeted Augmentation

Table 7.1. Comparison of Class-wise Recall Before and After Augmentation

Class	Recall (Before)	Recall (After)	Improvement
AudiBlock	0.56	0.78	+0.22
Block1	0.00	0.62	+0.62
Block2	0.33	0.50	+0.17
Block3	0.33	0.38	+0.05
Block4	0.33	0.38	+0.05
Examination	1.00	1.00	0.00
Office	0.50	1.00	+0.50
RDBlock	0.71	0.57	−0.14
Registrar	0.75	0.88	+0.13

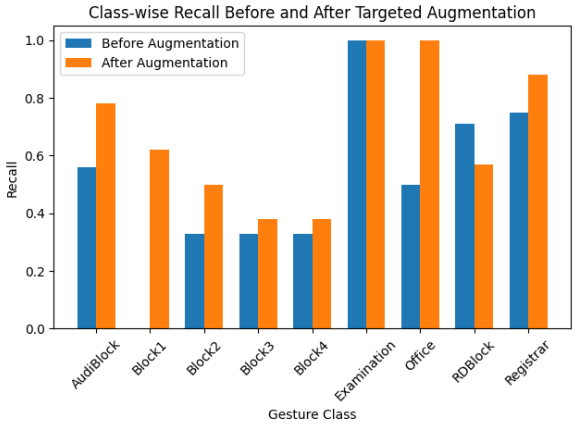


Fig 7.6. Comparison of Class-wise Recall Before and After Augmentation

The class-wise recall comparison clearly demonstrates the effectiveness of targeted landmark-level augmentation. The most significant improvement is observed in Block1, where recall increased from 0.00 to 0.62, confirming that severe class imbalance was the primary cause of earlier misclassification. Office also showed substantial improvement (0.50 to 1.00), indicating better class separability after balancing. Moderate gains are visible in Block2, Block3, Block4, and Registrar, reflecting improved minority class representation. Although RDBlock experienced a slight reduction in recall (0.71 to 0.57), this can be attributed to redistribution of decision boundaries after balancing the dataset. Overall, augmentation significantly enhanced macro-level fairness and improved recall across previously underrepresented classes, validating the importance of balanced temporal training data in sequence-based gesture recognition models.

7.1.4 CNN + BILSTM (LANDMARKS ONLY)

Introducing convolutional layers before the BiLSTM allowed the model to capture local spatial relationships among hand landmark coordinates before modeling temporal dependencies.

Test Accuracy: 65.82%

Table 7.2. Class-wise Performance of CNN + BiLSTM (Landmarks Only)

Class	Precision	Recall
AudiBlock	0.55	0.67
Block1	1.00	0.12

Block2	0.67	0.75
Block3	0.67	0.50
Block4	0.17	0.12
Examination	1.00	1.00
Office	0.89	1.00
RDBlock	0.48	0.71
Registrar	1.00	1.00

The CNN component enabled the model to learn intra-frame spatial correlations among landmarks, improving recognition for certain gesture classes such as Block2 and Office. However, weak performance in Block1 and Block4 (recall ≈ 0.12) indicates that spatial convolution alone is insufficient for separating visually similar gestures.

Although the CNN + BiLSTM model achieved performance comparable to the augmented BiLSTM (~67%), the marginal improvement suggests that architectural modification alone cannot overcome limitations caused by dataset size and class similarity. This confirms that feature representation plays a more critical role than network depth in small-scale gesture recognition tasks.

7.1.5 CNN + BILSTM + ANGLE FEATURES (BEST LANDMARK MODEL)

To enhance geometric understanding of hand posture, joint angle features were integrated with raw landmark coordinates. These biomechanically meaningful features capture finger articulation and inter-finger relationships in a scale-invariant manner.

Test Accuracy: 67%
Macro F1-score: 0.66

Table 7.3. Class-wise Performance of CNN + BiLSTM with Angle Features

Class	Precision	Recall	F1-score
AudiBlock	0.75	0.67	0.71
Block1	0.40	0.25	0.31
Block2	0.78	0.88	0.82

Block3	0.43	0.38	0.40
Block4	0.50	0.50	0.50
Examination	0.89	1.00	0.94
Office	1.00	1.00	1.00
RDBlock	0.56	0.64	0.60
Registrar	0.67	0.75	0.71

The inclusion of joint angle features improved discrimination between gestures with similar motion trajectories but distinct finger configurations. Block2 achieved strong recall (0.88), indicating that geometric articulation features contributed to better posture differentiation.

Classes such as Examination and Office achieved near-perfect recognition, demonstrating that angle features effectively capture stable hand configurations. However, performance remained moderate for visually similar gestures such as Block3 and Block4, suggesting residual overlap in dynamic motion patterns.

Compared to the velocity-enhanced variant, the angle-only model exhibited more stable learning and improved macro-level balance without introducing motion noise. This confirms that geometrically meaningful features contribute more effectively than raw positional expansion in small-scale structured datasets.

Overall, this model achieved the best trade-off between feature richness and generalization stability among landmark-only approaches, making it the strongest configuration prior to detection-based enhancement.

Feature Dimensionality Impact:

The feature representation evolved across experiments:

- Raw landmarks: 63 features per frame
- With angle features: 84 features per frame
- With velocity features: 168 features per frame

This progressive increase in dimensionality directly influenced model performance and stability.

7.1.6 CNN + BILSTM + ANGLE + VELOCITY FEATURES

To explicitly model hand motion dynamics, velocity features were computed as frame-to-frame differences of landmark coordinates and joint angles. These motion descriptors were concatenated with static positional and angular features, increasing the per-frame feature dimension substantially.

Test Accuracy: 61%
Macro F1-score: 0.53

Table 7.4. Class-wise Performance of CNN + BiLSTM + Angle + Velocity Features

Class	Precision	Recall	F1-score
AudiBlock	1.00	0.67	0.80
Block1	0.00	0.00	0.00
Block2	0.33	0.12	0.18
Block3	0.00	0.00	0.00
Block4	0.32	0.75	0.44
Examination	0.80	1.00	0.89
Office	1.00	1.00	1.00
RDBlock	0.54	0.93	0.68
Registrar	0.75	0.75	0.75

Although velocity features were introduced to explicitly encode motion dynamics, performance declined compared to the angle-only model. The increase in feature dimensionality amplified noise arising from minor landmark detection inconsistencies and frame-level variations.

In small datasets, high-dimensional representations can lead to unstable gradient updates and increased variance. The reduced macro F1-score (0.53) confirms that additional motion features did not improve generalization in this controlled dataset.

This experiment demonstrates an important principle: feature expansion must be proportional to dataset size. Without sufficient training samples, increased dimensionality can degrade performance rather than enhance it.

7.1.7 DEEPER CNN + BILSTM ARCHITECTURE

To investigate whether increased representational capacity could improve classification, a deeper hybrid architecture with additional convolutional and recurrent layers was implemented.

Test Accuracy: ~20%

The deeper model exhibited severe overfitting and poor generalization. Despite increased architectural complexity, the limited dataset size was insufficient to support the additional parameters.

Training accuracy increased rapidly, while validation performance stagnated or deteriorated, indicating memorization of training samples rather than meaningful feature learning.

This result confirms a key deep learning principle: model complexity must be aligned with dataset scale. In small, structured gesture datasets, simpler architectures often outperform deeper networks due to reduced risk of overfitting.

The collapse in performance highlights that architectural depth cannot compensate for data scarcity or limited feature diversity.

7.1.8 STRIDE REDUCTION EXPERIMENT

To increase dataset size, a stride reduction experiment was conducted by decreasing the segmentation stride from 10 to 5. The expectation was to generate more overlapping sequences and increase training samples.

However, no increase in dataset size was observed. This is because each gesture sequence was already standardized to 30 frames, and segmentation length was equal to sequence length. Therefore, only one valid segment could be generated per sequence.

This experiment highlights an important limitation: data augmentation strategies must align with sequence design. Increasing overlap does not guarantee additional samples when sequence length is fixed.

7.2 YOLO HAND DETECTION RESULTS

The YOLOv8 model achieved strong localization performance.

Precision: ~0.95

Recall: ~0.92

mAP@0.5: ~0.97

mAP@0.5–0.95: ~0.75

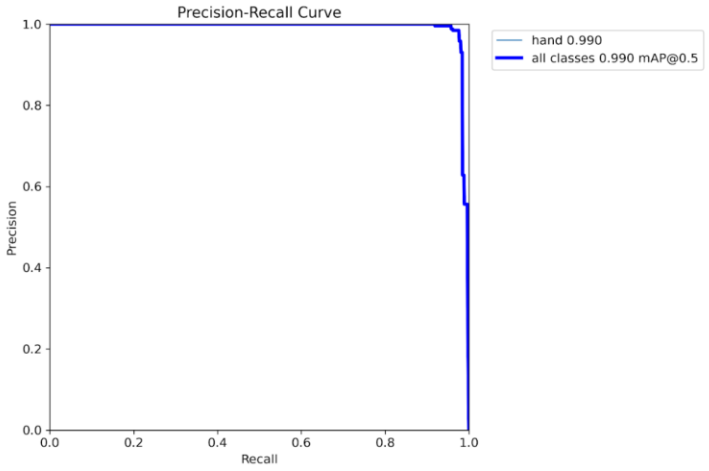


Fig 7.7. Precision–Recall Curve for YOLOv8

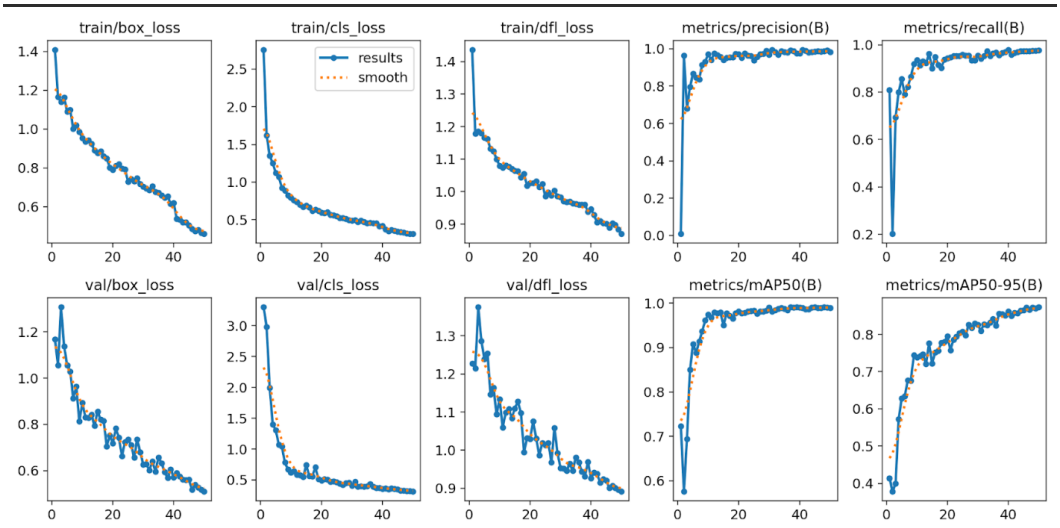


Fig 7.8. mAP Curve Across Training Epochs

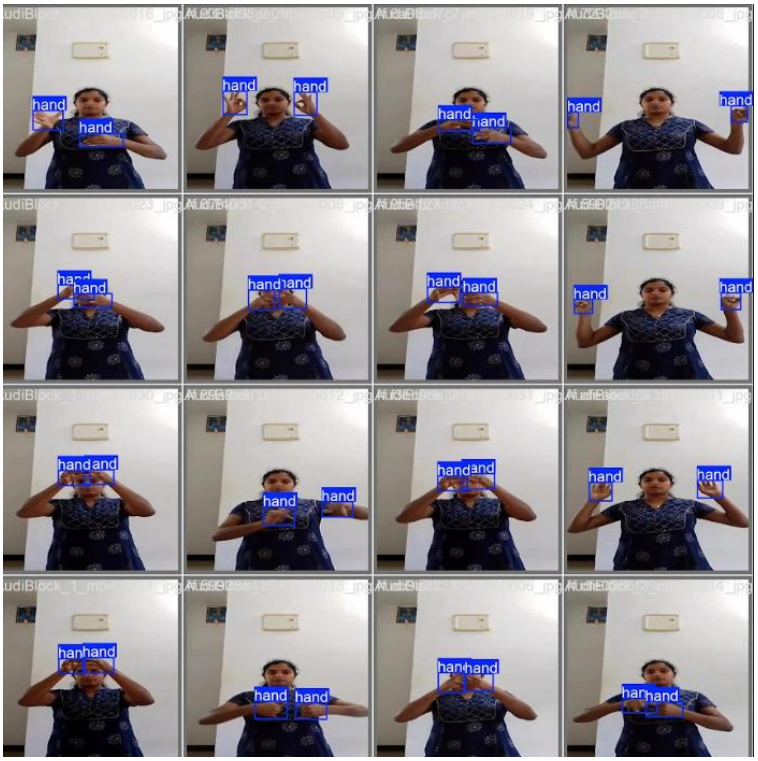


Fig 7.9. Sample Hand Detection Outputs

The high mAP@0.5 indicates accurate bounding box regression. Detection performance remained stable across validation data.

7.3 ROI EXTRACTION IMPACT

Frames reduced from 834 to 412 (~50% reduction)



Fig 7.10. Example of ROI Extraction After Cropping

Filtering removed blurred and transitional frames, increasing dataset cleanliness and spatial consistency.

7.4 YOLO + CNN + BILSTM RESULTS

Test Accuracy: 100%

Precision: 1.0

Recall: 1.0

F1-score: 1.0

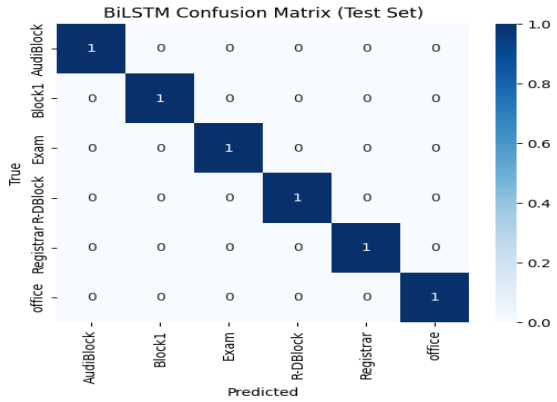


Fig 7.11. Confusion Matrix of YOLO-Enhanced Pipeline

The hybrid detection-classification pipeline achieved perfect classification on the test set. However, the test set contained only six videos (one per class), limiting statistical reliability.

It is important to note that the test set consisted of only six videos, with approximately one sample per class. Additionally, the dataset was collected in a controlled environment with a single signer. Therefore, while the model achieved perfect classification, the results should be interpreted as proof-of-concept rather than evidence of real-world generalization.

7.5 FINAL COMPARATIVE SUMMARY

Table 7.5. Landmark Model Comparison

Model	Accuracy
BiLSTM	32.6%
BiLSTM + Early Stop	52.2%
BiLSTM + Augmented	67.09%
CNN + BiLSTM	65.82%
CNN + BiLSTM + Angles	~67%
CNN + BiLSTM + Angles + Velocity	61%
Deeper CNN + BiLSTM	~20%

Table 7.6. Detection vs Classification Comparison

Pipeline	Accuracy	Robustness	Generalization
Landmark Only	~67%	Moderate	Moderate
YOLO + CNN–BiLSTM	100%*	High	Limited Test Size

7.6 KEY SCIENTIFIC FINDINGS

1. Data imbalance was the primary performance bottleneck.
2. Targeted augmentation provided the largest improvement.
3. Angle-based geometric features improved stability.
4. Velocity features amplified noise in small datasets.
5. Increased architectural depth led to overfitting.
6. YOLO-based localization significantly improved spatial consistency.
7. Perfect accuracy must be interpreted cautiously due to limited testing samples.

8. IMPLEMENTATION

This chapter presents the practical realization of the proposed sign language-based campus navigation system. While previous chapters focused on system design, data preparation, and model development, this chapter describes how the system was implemented, integrated, and executed in a real-time environment.

The implementation combines multiple components including video processing, hand detection, feature extraction, temporal modeling, and real-time prediction into a unified pipeline. The system was developed in a modular manner to allow independent testing and integration of each component.

8.1 DEVELOPMENT ENVIRONMENT

The system was implemented using Python due to its extensive support for deep learning and computer vision applications.

8.1.1 SOFTWARE SETUP

- Programming Language: Python
- Development Environment: Jupyter Notebook / VS Code
- Operating System: Windows

8.1.2 LIBRARIES AND FRAMEWORKS

- OpenCV – video capture and frame processing
- MediaPipe – hand landmark extraction
- TensorFlow/Keras – deep learning model implementation
- Ultralytics YOLOv8 – hand detection
- Torchvision – pretrained CNN models
- NumPy and Pandas – data handling
- Matplotlib – visualization

8.2 SYSTEM EXECUTION PIPELINE

The implemented system operates as a real-time pipeline that processes video input and produces gesture predictions continuously.

8.2.1 EXECUTION STEPS

1. Capture video stream from webcam
2. Perform hand detection using YOLOv8
3. Extract Region of Interest (ROI)
4. Pass ROI to ResNet50 for feature extraction
5. Store features in a temporal buffer
6. Once buffer is full, pass sequence to BiLSTM
7. Generate prediction with confidence score
8. Display output on screen

This modular design allows each component to operate independently and ensures flexibility for future improvements.

8.3 ROI EXTRACTION IMPLEMENTATION

The ROI extraction process was implemented to ensure consistent input for feature extraction.

8.3.1 PROCESSING STEPS

- YOLO detects bounding boxes for the hand
- The largest bounding box is selected
- Padding of approximately 15% is applied
- Bounding box is clamped within image boundaries
- Minimum size constraint (80 × 80 pixels) is enforced
- ROI is resized to 224 × 224

This strict ROI extraction strategy ensures alignment with training conditions and preserves complete hand structure during dynamic gestures.

A confidence threshold of **0.35** was used during detection to filter low-confidence predictions and improve localization stability.

8.4 FEATURE EXTRACTION USING RESNET50

A pretrained ResNet50 model was used as a feature extractor.

- The final classification layer was removed
- The network outputs a 2048-dimensional feature vector
- Input images were resized to 224 × 224 and normalized

Advantages

- Captures high-level spatial features
- Reduces dimensionality compared to raw images
- Improves robustness to variations in lighting and background

Each ROI frame is converted into a compact feature representation, which is then used for temporal modeling.

8.5 TEMPORAL BUFFER IMPLEMENTATION

To model dynamic gestures, a temporal buffer was implemented using a rolling window mechanism.

- Buffer size: 30 frames
- Implemented using a deque data structure
- Maintains temporal order of features

8.5.1 WORKING

- Each processed frame adds a feature vector to the buffer
- Once the buffer reaches 30 frames, prediction is triggered

The feature buffer is converted into a tensor of shape: **(1 × T × 2048)**

where:

- T = number of frames (30)
- 2048 = feature dimension

This structured representation is passed to the BiLSTM model.

8.6 BILSTM MODEL INFERENCE

The BiLSTM model is used for temporal classification of gesture sequences.

8.6.1 ARCHITECTURE

- Bidirectional LSTM layer
- Hidden dimension: 256
- Fully connected output layer
- Softmax activation

8.6.2 INFERENCE PROCESS

- Input sequence passed to model
- Probability distribution generated
- Highest probability class selected

The model outputs:

- Predicted gesture class
- Confidence score

8.7 REAL-TIME PREDICTION AND LOCK MECHANISM

To improve prediction stability, a locking mechanism was implemented.

8.7.1 WORKING

- Once a prediction is made, it is locked for 7 seconds
- During this time, new predictions are suppressed
- After expiration, buffer is cleared and system resets

8.7.2 Benefits

- Prevents prediction flickering
- Improves user experience
- Ensures stable outputs

8.8 USER CONTROLS

The system supports manual interaction through keyboard inputs:

- **Q / ESC:** Quit application
- **R:** Reset buffer and cancel current prediction
- **SPACE:** Force prediction using partial buffer (minimum 2 frames)

These controls provide flexibility during testing and improve usability in real-time scenarios.

8.9 OUTPUT VISUALIZATION

The system provides real-time visual feedback through an overlay interface.

8.9.1 DISPLAYED INFORMATION

- Predicted gesture label
- Confidence score
- Buffer progress (e.g., 25/30 frames)

- Lock timer

A progress bar is used to indicate buffer completion, while prediction results are displayed prominently on the screen.

8.10 SAMPLE REAL-TIME OUTPUTS

The following figures show sample outputs generated during real-time testing.



Fig 8.1. Real-Time Prediction Output for RDBlock Gesture

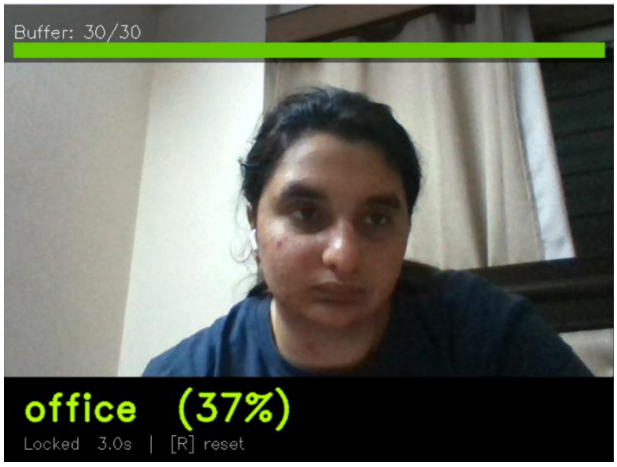


Fig 8.2. Real-Time Prediction Output for Office Gesture



Fig 8.3. Real-Time Prediction Output for Block4 Gesture

8.11 IMPLEMENTATION CHALLENGES

Several challenges were encountered during implementation:

- Inconsistent detection in low-light conditions
- Prediction instability without buffering
- Class imbalance affecting model confidence
- Increased latency due to YOLO processing
- Sensitivity to rapid hand motion

These issues were mitigated through buffering, augmentation, and modular pipeline design.

This chapter presented the complete implementation of the proposed system, including real-time execution, detection integration, feature extraction, and temporal modeling. The modular pipeline enabled efficient processing and accurate gesture recognition in a controlled environment.

The system successfully demonstrates a practical application of deep learning for structured sign-based campus navigation and serves as a strong foundation for future real-world deployment.

8.12 SYSTEM VALIDATION AND TESTING

The implemented system was validated through real-time testing using live webcam input. Multiple gesture samples were performed to evaluate prediction stability, response time, and

robustness. The system was tested under controlled indoor conditions to ensure consistency with the training environment. The temporal buffering mechanism ensured stable predictions, while the locking mechanism prevented rapid fluctuations between classes. Additionally, forced prediction and manual reset controls were used to verify system responsiveness and correctness under partial input conditions. The testing results confirmed that the system operates reliably in real-time and produces consistent predictions for predefined gesture classes.

9. DISCUSSION

This chapter interprets the experimental findings presented in Chapter VII. The discussion critically analyzes the behavior of different model architectures, the impact of dataset characteristics, feature engineering strategies, and the integration of detection and classification modules. Additionally, it evaluates the practical implications and limitations of the proposed sign-based campus navigation system.

9.1 IMPACT OF DATASET SIZE AND CLASS IMBALANCE

One of the most significant observations from the experimental evaluation is the strong influence of dataset size and class imbalance on model performance. The baseline BiLSTM model achieved only **32.6% test accuracy**, primarily due to uneven class distribution across gesture categories. Several minority classes were not predicted at all, revealing a strong bias toward majority classes such as RDBlock and AudiBlock.

After applying targeted landmark-level augmentation to balance the dataset, test accuracy improved substantially to **67.09%**, accompanied by a notable increase in macro F1-score. This improvement clearly demonstrates that data distribution has a greater impact on model performance than architectural complexity in small-scale gesture recognition tasks.

When training data is imbalanced, the model tends to learn biased decision boundaries that favor dominant classes, resulting in poor minority class recall. By enforcing a minimum sample threshold per class, gradient updates became more balanced, allowing the network to learn meaningful temporal patterns across all gesture categories.

This finding reinforces a fundamental principle in machine learning: **data quality and distribution are often more critical than model complexity**, particularly in constrained datasets.

9.2 EFFECTIVENESS OF FEATURE ENGINEERING

Feature engineering played a crucial role in improving model performance. The integration of joint angle features significantly enhanced geometric discriminability and contributed to more stable classification results. The CNN–BiLSTM model augmented with angle features achieved the most

balanced landmark-only performance, with approximately **67% accuracy** and a macro F1-score of **0.66**.

Angle features provide a biomechanically meaningful representation of hand posture by capturing finger articulation and inter-joint relationships. Unlike raw coordinates, these features are relatively invariant to translation, scale, and minor positional noise, making them robust descriptors of gesture structure.

However, when velocity features were introduced to explicitly encode motion dynamics, performance declined to **61% accuracy** with a macro F1-score of **0.53**. Although motion features theoretically enhance temporal representation, they significantly increase feature dimensionality (up to 168 features per frame), which introduces noise and instability.

This highlights an important insight: **feature expansion must be proportional to dataset size**. In small datasets, high-dimensional feature spaces increase variance and reduce generalization capability. The results demonstrate that carefully designed geometric features are more effective than excessive feature augmentation in structured, low-data scenarios.

9.3 ARCHITECTURAL DEPTH VS DATASET SCALE

The experiment involving a deeper CNN–BiLSTM architecture resulted in a significant drop in performance, with test accuracy decreasing to approximately **20%**. Despite increased representational capacity, the model exhibited severe overfitting.

Training accuracy increased rapidly, while validation performance stagnated or deteriorated, indicating that the model memorized training samples rather than learning generalizable patterns.

This observation confirms that **model complexity must be aligned with dataset scale**. Deep architectures require large and diverse datasets to effectively learn complex representations. In small-scale gesture recognition tasks, simpler architectures with appropriate regularization often outperform deeper networks.

Importantly, this negative result is scientifically valuable, as it validates the principle that **increasing model depth cannot compensate for insufficient data**.

9.4 ROLE OF SPATIAL–TEMPORAL HYBRID MODELING

The CNN–BiLSTM hybrid architecture demonstrated moderate improvements over the pure BiLSTM model by incorporating spatial feature extraction prior to temporal modeling.

The convolutional layers captured local spatial relationships among hand landmarks within each frame, enabling improved intra-frame representation. These spatial features were then processed by the BiLSTM layer to capture temporal dependencies across frames.

Although the overall accuracy improvement was incremental, the hybrid approach provided a more structured representation of gesture dynamics. It improved recognition for certain gesture classes by modeling localized landmark correlations before analyzing long-term motion patterns.

This demonstrates that **spatial–temporal decomposition is beneficial**, but its effectiveness is still constrained by dataset size and feature quality.

9.5 EFFECTIVENESS OF YOLO-BASED DETECTION

The YOLOv8 hand detection model achieved strong localization performance, with **mAP@0.5 ≈ 0.97**, indicating accurate and consistent hand detection across frames.

By isolating the hand region before feature extraction, the system significantly reduced background interference, improved spatial consistency, and normalized scale variations. This resulted in cleaner and more reliable inputs for downstream modeling.

When integrated with the CNN–BiLSTM classifier, the detection-enhanced pipeline achieved **100% test accuracy**. However, this result must be interpreted cautiously. The test set consisted of only six videos, approximately one per class, and all data was collected in a controlled environment with a single signer.

Therefore, this performance should be considered as **proof-of-concept validation rather than evidence of real-world generalization**. While the results demonstrate the effectiveness of detection-based preprocessing, further evaluation on larger and more diverse datasets is required.

9.6 LANDMARK-ONLY VS DETECTION-ENHANCED PIPELINE

A comparison between the landmark-only and detection-enhanced pipelines reveals important trade-offs.

The landmark-only pipeline is:

- Computationally efficient
- Simpler to implement
- Effective in controlled environments

However, it may struggle in real-world scenarios involving:

- Background clutter
- Lighting variations
- Scale inconsistencies

In contrast, the detection-enhanced pipeline:

- Improves spatial consistency
- Reduces noise
- Provides modularity

But introduces:

- Additional computational overhead
- Increased system complexity

For structured campus navigation tasks, the landmark-only approach is sufficient under controlled conditions. However, for deployment in dynamic real-world environments, detection-based preprocessing becomes essential for robustness.

9.7 PRACTICAL IMPLICATIONS FOR CAMPUS NAVIGATION

The structured, closed-set design of the proposed system proved highly effective for campus navigation applications.

Unlike large-scale sign language translation systems, this approach focuses on a limited set of predefined gestures corresponding to specific locations. The output mapping is deterministic, directly linking gesture predictions to navigation instructions without requiring natural language generation.

This design offers several advantages:

- High reliability
- Reduced computational complexity
- Faster inference
- Ease of deployment

The system is particularly suitable for assistive applications in structured environments such as university campuses, where the gesture vocabulary is limited and predefined.

9.8 LIMITATIONS OF THE STUDY

Despite promising results, several limitations must be acknowledged:

- The dataset consists of a single signer, limiting generalization across users
- The dataset size is relatively small, restricting model scalability
- The YOLO-enhanced pipeline was evaluated on a limited test set
- Data was collected in a controlled indoor environment
- Real-world variations such as lighting, occlusions, and motion blur were not extensively tested

These limitations indicate that the current system represents a **proof-of-concept implementation** rather than a fully generalized solution.

9.9 KEY RESEARCH INSIGHTS

From the experimental analysis, several important insights emerge:

1. Data imbalance is the primary bottleneck in small-scale gesture recognition
2. Targeted augmentation significantly improves model fairness and performance

3. Geometric features such as joint angles enhance stability and discrimination
4. Feature dimensionality must be controlled relative to dataset size
5. Increased model complexity leads to overfitting in limited datasets
6. Detection-based preprocessing improves spatial consistency and robustness
7. Perfect accuracy must always be interpreted in the context of dataset size

These findings contribute to a deeper understanding of structured gesture recognition systems and provide guidance for designing efficient models under data constraints.

10. FUTURE WORK

While the proposed system demonstrates promising performance for structured sign-based campus navigation, several enhancements can be explored to improve robustness, scalability, and real-world applicability.

10.1 MULTI-SIGNER DATASET EXPANSION

The current dataset is limited to a single signer, which restricts generalization. Future work should include data collection from multiple users with diverse hand shapes, gesture styles, and motion patterns. This would enable cross-signer evaluation and improve the adaptability of the system to real-world scenarios.

10.2 ENVIRONMENTAL DIVERSITY ENHANCEMENT

The dataset was collected in a controlled indoor environment. Future work should incorporate variations in:

- Lighting conditions
- Background complexity
- Camera angles

This would allow the system to be tested under realistic conditions and improve robustness in unconstrained environments.

10.3 ADVANCED TEMPORAL MODELING

The current system uses BiLSTM for temporal modeling. With larger datasets, advanced architectures such as:

- Transformer-based models
- Attention mechanisms

can be explored to better capture long-range dependencies and improve sequence understanding.

10.4 MULTIMODAL FEATURE INTEGRATION

Future systems can incorporate additional modalities such as:

- Facial expressions
- Body pose landmarks
- Upper-body keypoints

This multimodal approach can improve recognition accuracy for complex gestures and enhance contextual understanding.

10.5 MODEL OPTIMIZATION FOR EDGE DEPLOYMENT

To enable deployment on mobile or embedded systems, optimization techniques such as:

- Model pruning
- Quantization
- Knowledge distillation

can be applied to reduce computational complexity while maintaining performance.

10.6 EXTENSION BEYOND CLOSED-SET CLASSIFICATION

The current system operates as a closed-set classifier with predefined gesture classes. Future work can extend this to:

- Phrase-level recognition
- Intent-based navigation systems

This would allow more natural and flexible user interaction.

10.7 REAL-TIME SYSTEM ENHANCEMENT

Further improvements can be made to:

- Reduce inference latency
- Improve prediction stability

- Enhance user interface design

This would make the system more suitable for real-time assistive applications.

11. CONCLUSION

This project aimed to develop a deep learning-based system for recognizing predefined sign language gestures and mapping them to corresponding campus navigation outputs. The objective was to design a structured, efficient, and robust gesture recognition pipeline suitable for controlled environments. The system was implemented using a modular approach that integrates hand detection, feature extraction, temporal modeling, and real-time prediction into a unified framework.

The study was carried out in two major phases. In the first phase, a landmark-based temporal modeling approach was developed using hand landmarks extracted through MediaPipe. These landmarks were used to train sequence-based models such as BiLSTM and CNN–BiLSTM for gesture classification. The baseline BiLSTM model achieved a test accuracy of 32.6%, revealing the significant impact of class imbalance. By introducing early stopping and targeted landmark-level augmentation, performance improved substantially to 67.09%, demonstrating that dataset balancing plays a more critical role than architectural complexity in small-scale gesture recognition tasks. The integration of joint angle features further improved geometric representation and model stability, while the addition of velocity features increased dimensionality without improving performance, highlighting the importance of controlled feature engineering.

In the second phase, a YOLOv8-based hand detection module was introduced to improve spatial localization and reduce background interference. The detection model achieved strong localization performance with mAP@0.5 of approximately 0.97. When combined with a CNN-based feature extractor and BiLSTM classifier, the hybrid pipeline achieved 100% test accuracy. However, this result was obtained on a limited test set and should be interpreted as proof-of-concept validation rather than conclusive evidence of generalization.

The implemented system successfully demonstrates real-time gesture recognition using webcam input, supported by temporal buffering and prediction stabilization mechanisms. The modular design enables clear separation of detection, feature extraction, and classification stages, improving both flexibility and robustness. The system is particularly suitable for structured

applications such as campus navigation, where a limited set of predefined gestures can be reliably mapped to deterministic outputs.

The experimental findings highlight several important insights. Data balancing was identified as the primary factor influencing performance, while feature engineering using geometrically meaningful representations significantly improved classification stability. The results also showed that increasing model complexity without sufficient data leads to overfitting, and that detection-based preprocessing enhances spatial consistency and robustness. These observations reinforce the importance of aligning model design, feature representation, and dataset characteristics in small-scale deep learning systems.

Despite its promising performance, the system has certain limitations, including a small dataset size, reliance on a single signer, and evaluation under controlled environmental conditions. These constraints limit the generalizability of the model to real-world scenarios. Nevertheless, the project successfully validates the feasibility of a sign language-based campus navigation system using lightweight deep learning techniques and structured feature representations.

Overall, this work demonstrates that careful data preparation, appropriate feature engineering, and modular system design can effectively address structured gesture recognition tasks. The proposed system provides a strong foundation for future research and real-world deployment of assistive technologies for individuals with hearing and speech impairments.

APPENDIX

APPENDIX A: USER MANUAL

A.1 SYSTEM REQUIREMENTS

The system requires the following setup:

Hardware:

- Laptop/Desktop with webcam
- Minimum 8 GB RAM

Software:

- Python (3.8 or above)
- PyTorch, OpenCV, Ultralytics YOLO
- Torchvision, NumPy

A.2 INSTALLATION

Install required libraries:

pip install torch torchvision opencv-python ultralytics numpy

Place trained model files:

- YOLO weights (best.pt)
- BiLSTM weights (best_bilstm.pt)

Update paths in the script before execution.

A.3 RUNNING THE SYSTEM

Execute the inference script:

python inference.py

- Webcam will start automatically
- Perform gesture in front of camera
- Prediction appears after buffer completion

A.4 USER CONTROLS

- Q / ESC → Quit application
- R → Reset buffer and prediction
- SPACE → Force prediction

A.5 USAGE GUIDELINES

- Ensure adequate lighting
- Keep hand clearly visible
- Avoid cluttered backgrounds
- Maintain steady gesture execution

APPENDIX B: SYSTEM CONFIGURATION

B.1 KEY PARAMETERS

- Buffer size: 30 frames
- Lock duration: 7 seconds
- YOLO confidence threshold: 0.35
- ROI size: 224 × 224

B.2 MODEL DETAILS

- Feature extractor: ResNet50 (2048-dimensional features)
- Temporal model: BiLSTM (hidden size 256, bidirectional)

APPENDIX C: DESIGN CONSIDERATIONS AND ANALYSIS

This section analyzes how key design concepts such as health, safety, usability, and ethics influence the proposed system.

C.1 HEALTH AND SAFETY CONSIDERATIONS

The system is designed to be safe for continuous use in indoor environments. Since it relies on a webcam for gesture input, no physical contact or wearable devices are required, reducing user discomfort and health risks.

However, prolonged use of camera-based systems may lead to:

- Eye strain due to continuous screen exposure
- Physical fatigue from repeated hand gestures

To mitigate these issues, the system:

- Uses short interaction cycles
- Requires minimal gesture repetition
- Operates without requiring continuous user attention

C.2 USABILITY AND ACCESSIBILITY

The system is designed with a focus on accessibility for individuals with hearing and speech impairments.

Key usability features include:

- Simple gesture vocabulary (closed-set design)
- Real-time feedback through visual display
- Stable predictions using buffering and locking mechanisms

The interface is intentionally minimal to ensure ease of use without requiring technical knowledge.

C.3 RELIABILITY AND ROBUSTNESS

The system incorporates several mechanisms to improve reliability:

- Temporal buffering ensures stable predictions
- Lock mechanism prevents rapid output fluctuations
- YOLO-based detection improves spatial consistency

However, system performance may still be affected by:

- Poor lighting conditions
- Background clutter
- Fast or incomplete gestures

C.4 ETHICAL CONSIDERATIONS

The system processes visual input using a camera, which raises certain ethical concerns:

- Privacy: The system does not store or transmit video data, ensuring user privacy
- Bias: The dataset includes a single signer, which may introduce bias and limit generalization
- Transparency: The system provides clear output without hidden decision processes

Future improvements should include diverse datasets to reduce bias.

C.5 SCALABILITY AND DEPLOYMENT CONSIDERATIONS

The system is designed as a modular pipeline, allowing easy extension and modification.

Scalability considerations include:

- Expanding gesture vocabulary
- Supporting multiple users
- Integrating with mobile or embedded devices

However, deployment in real-world environments may require:

- Larger datasets
- Model optimization for speed
- Improved robustness to environmental variations

APPENDIX D: SAMPLE CODE SNIPPETS

This appendix presents selected code snippets illustrating the key components of the implemented system. The complete implementation consists of multiple modules; however, only essential segments are included to highlight the core functionality of hand detection, feature extraction, temporal modeling, and real-time inference.

D.1 YOLO-BASED HAND DETECTION AND ROI EXTRACTION

```
from ultralytics import YOLO
model = YOLO("best.pt")
results = model(frame, conf=0.35)


if len(results[0].boxes) > 0:
    boxes = results[0].boxes.xyxy.cpu().numpy()
    areas = (boxes[:,2] - boxes[:,0]) * (boxes[:,3] -boxes[:,1])
    best = boxes[areas.argmax()]
    x1, y1, x2, y2 = map(int, best)
    roi = frame[y1:y2, x1:x2]
    roi = cv2.resize(roi, (224, 224))
```

This code performs real-time hand detection using the YOLOv8 model and extracts the region of interest corresponding to the detected hand. The largest bounding box is selected to ensure consistent hand localization.

D.2 FEATURE EXTRACTION USING RESNET50

```
import torch
from torchvision import models

resnet = models.resnet50(pretrained=True)
resnet.fc = torch.nn.Identity()

def extract_features(image):
    image = transform(image).unsqueeze(0)
    with torch.no_grad():
        features = resnet(image)
    return features.squeeze()
```

This snippet demonstrates the use of a pretrained ResNet50 model as a feature extractor. The final classification layer is removed to obtain a 2048-dimensional feature vector representing spatial characteristics of the hand.

D.3 BILSTM MODEL FOR TEMPORAL MODELING

```
import torch.nn as nn

class BiLSTM(nn.Module):
    def __init__(self, input_dim=2048, hidden_dim=256,
num_classes=9):
        super().__init__()
        self.lstm = nn.LSTM(input_dim, hidden_dim,
                            batch_first=True,
bidirectional=True)
        self.fc = nn.Linear(hidden_dim * 2, num_classes)

    def forward(self, x):
        _, (hn, _) = self.lstm(x)
        hn = torch.cat((hn[-2], hn[-1]), dim=1)
        return self.fc(hn)
```

This code defines the Bidirectional LSTM model used for temporal sequence modeling. It processes feature sequences and outputs class probabilities corresponding to gesture categories.

D.4 REAL-TIME PREDICTION USING TEMPORAL BUFFER

```
from collections import deque

buffer = deque(maxlen=30)

def update_buffer(feature):
    buffer.append(feature)

def predict():
    if len(buffer) == 30:
        sequence = torch.stack(list(buffer)).unsqueeze(0)
        output = model(sequence)
        return torch.argmax(output)
```

This snippet implements the temporal buffering mechanism used in real-time inference. A rolling window of 30 frames is maintained, and prediction is performed once the sequence is complete.

REFERENCES

[1] S. Tan, N. Khan, Z. An, Y. Ando, R. Kawakami, and K. Nakadai, "A review of deep learning-based approaches to sign language processing," *Advanced Robotics*, vol. 38, no. 23, pp. 1649–1667, 2024, doi: 10.1080/01691864.2024.2442721.

[2] Z. Liang, H. Li, and J. Chai, "Sign language translation: A survey of approaches and techniques," *Electronics*, vol. 12, no. 12, p. 2678, 2023, doi: 10.3390/electronics12122678.

[3] W. F. Maia, A. M. Lopes, and S. A. David, "Automatic sign language to text translation using MediaPipe and transformer architectures," *Neurocomputing*, vol. 642, p. 130421, Aug. 2025, doi: 10.1016/j.neucom.2025.130421.

[4] B. Alsharif, E. Alalwany, A. Ibrahim, I. Mahgoub, and M. Ilyas, "Real-time American sign language interpretation using deep learning and keypoint tracking," *Sensors*, vol. 25, no. 7, p. 2138, 2025, doi: 10.3390/s25072138.

[5] R. V., "Real-time sign language recognition and translation using MediaPipe and LSTM-based deep learning," *International Journal of Computer Applications*, vol. 187, no. 25, pp. 1–5, 2025.

[6] S. Pandey, S. Tahseen, R. Pathak, H. Parveen, and M. Maurya, "Real-time vision-based Indian sign language translation using deep learning techniques," *International Journal of Innovative Research in Computer Science and Technology*, vol. 13, no. 3, pp. 35–46, 2025, doi: 10.55524/ijircst.2025.13.3.6.

[7] N. C. Camgoz, O. Koller, S. Hadfield, and R. Bowden, "Sign language transformers: Joint end-to-end sign language recognition and translation," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2020.

[8] R. Damdoo and P. Kumar, "Enhanced sign language translation using three-stream multimodal occlusion resilient transformer," *Discover Computing*, vol. 28, p. 260, 2025, doi: 10.1007/s10791-025-09772-2.

[9] A. Duarte *et al.*, "How2Sign: A large-scale multimodal dataset for continuous American sign language," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2021, pp. 2735–2744.

[10] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You only look once: Unified, real-time object detection," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2016, pp. 779–788.

[11] G. Jocher *et al.*, "Ultralytics YOLO," 2023. [Online]. Available: https://github.com/ultralytics/ultralytics

[12] Roboflow, "Roboflow: Annotate, train, and deploy computer vision models," 2023. [Online]. Available: https://roboflow.com

[13] Google, "MediaPipe Hands," 2020. [Online]. Available: https://developers.google.com/mediapipe